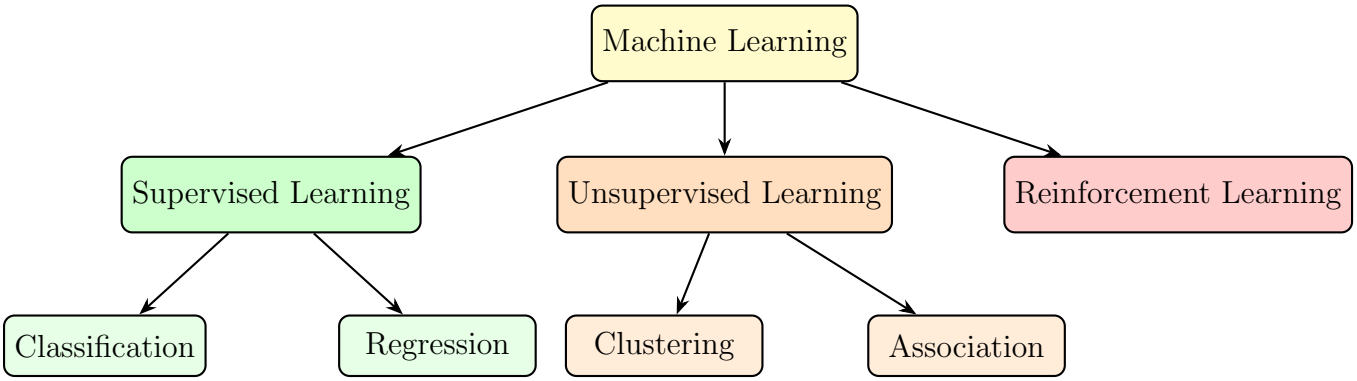
Chapter 9: Unsupervised Learning

April 2, 2026



1 Objective of Unsupervised Learning

Unsupervised learning deals with analysing **unlabelled data** in order to discover hidden patterns, structures, or relationships without any prior training or predefined output labels.

- No output variable or class label is provided.
- Emphasizes **knowledge discovery** rather than direct prediction.
- **Major techniques include:**
  - Clustering
  - Association Rule Learning

2 Unsupervised vs Supervised Learning

Aspect	**Supervised Learning**	**Unsupervised Learning**
Data Type	Labeled	Unlabeled
Objective	Predict outcome	Discover structure
Output	Class / Value	Groups / Rules
Examples	Regression, Classification	Clustering, Association

Statistical View:

- **Supervised learning** estimates the posterior probability $P(Y \mid X)$, where X represents input features and Y denotes the target label.
- **Unsupervised learning** focuses on modelling the joint probability $P(X, Y)$ in order to understand the underlying structure of the data.

Simple Example:

- In **supervised learning**, given emails (X) with known labels $Y \in \{\text{Spam}, \text{Not Spam}\}$, the model learns $P(Y \mid X)$ to predict whether a new email is spam.
- In **unsupervised learning**, only the email features (X) are available, and the model groups similar emails together by analysing $P(X, Y)$ without knowing the actual labels in advance.

3 Applications of Unsupervised Learning

- Customer and market segmentation: Grouping customers based on purchasing behavior to target marketing strategies effectively.
- Fraud and anomaly detection: Identifying unusual patterns in transactions or system behavior to prevent fraud.
- Image processing and face recognition: Analyzing images to detect patterns, classify objects, or recognize faces without labeled data.
- Gene analysis and bioinformatics: Discovering hidden structures and relationships in genetic data for research and medical applications.
- Dimensionality reduction: Reducing large datasets to key features to simplify analysis and improve computation efficiency.
- Document clustering: Grouping similar documents or articles for topic analysis and information retrieval.
- Artificial Intelligence systems: Enabling AI to learn patterns and make decisions without explicit supervision.

4 Clustering

4.1 Definition

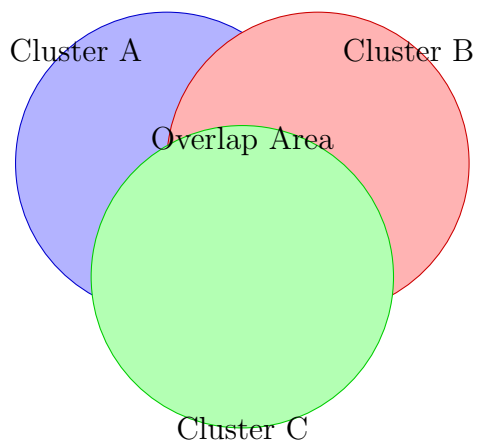
Clustering is the process of grouping objects such that:

- Objects within a cluster are highly similar
- Objects from different clusters are highly dissimilar

4.2 Key Characteristics

- Unsupervised learning task
- No predefined class labels
- Reduces data complexity
- Helps discover hidden patterns in data
- Flexible with different types of data (numerical, categorical, text)
- Scalable to large datasets using efficient algorithms
- Basis for other techniques like anomaly detection and recommendation systems

4.3 Clustering Diagram

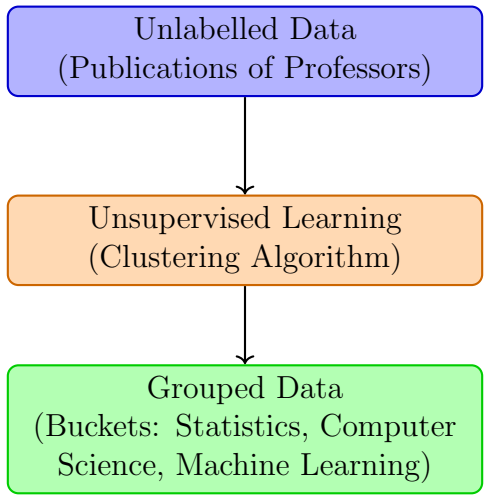


Explanation: The diagram shows three clusters A, B, and C. Objects inside each circle are similar, and the overlap areas indicate objects that share characteristics of multiple clusters.

4.4 Applications of Clustering

- Text Data Mining: Grouping similar documents, articles, or social media posts to identify topics or summarize content.
- Customer Segmentation: Grouping customers based on purchasing behavior, preferences, or demographics for targeted marketing strategies.
- Anomaly Detection: Identifying unusual patterns or outliers in data such as fraud detection, network intrusion, or faulty sensor readings.

4.5 Flow Diagram of Analysis



4.6 Example: Clustering Professors Based on Research Publications

You are invited to take a session on **Machine Learning** for the induction of professors at a reputed university. To gauge their familiarity with the subject without directly asking, you analyze **publicly available data** on their research publications. Since Machine Learning lies at the intersection of **Statistics** and **Computer Science**, you collect the number of publications of each professor in these two areas.

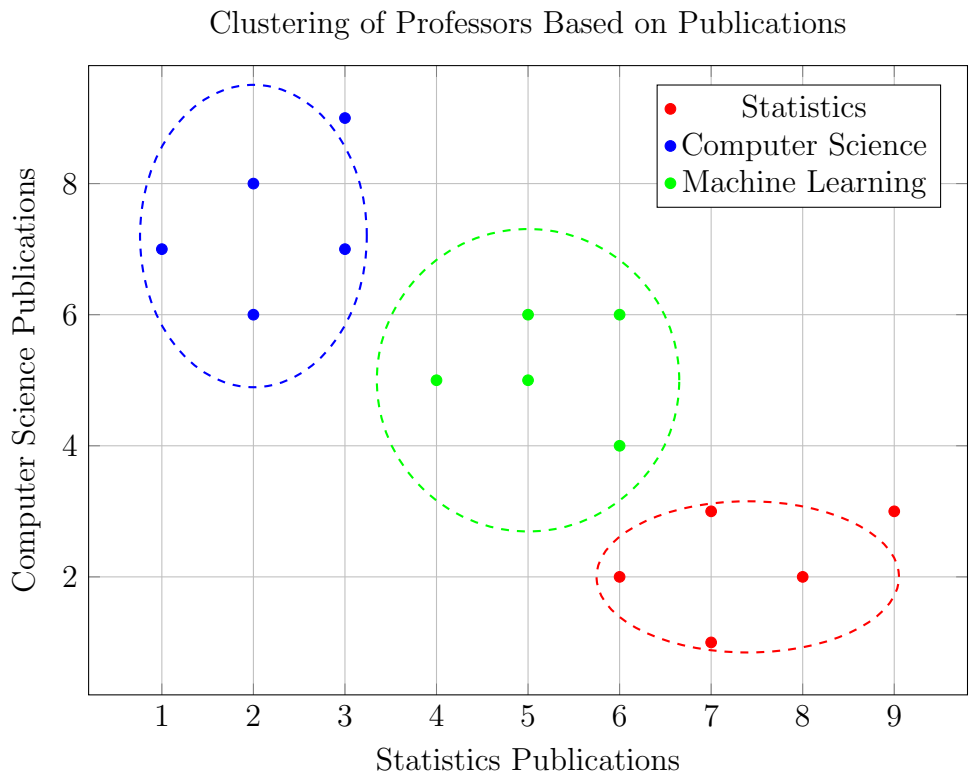
By applying an **unsupervised learning algorithm (clustering)**, you can group the professors into three clusters:

- Statistics experts
- Computer Science experts
- Machine Learning experts

Aim of clustering:

- To identify natural groupings among professors based on their publication profiles.
- To infer expertise levels in each domain without prior labels.
- To guide your session by tailoring content according to professors' strengths and knowledge gaps.

Clusters for the conference attendees



4.7 Explanation

By plotting the number of publications in **Statistics** (x-axis) and **Computer Science** (y-axis), you can visualize clusters representing professors' expertise:

- Red points: Professors specializing in Statistics.
- Blue points: Professors specializing in Computer Science.
- Green points: Professors whose work spans both areas, i.e., Machine Learning.

This clustering helps infer the expertise of the professors **without direct questioning** and allows you to **tailor your session accordingly**.

Method	Characteristics
Partitioning Methods	• Divide data into non-overlapping subsets (clusters) • Each object belongs to exactly one cluster • Requires number of clusters to be specified in advance • Example: K-Means, K-Medoids
Hierarchical Methods	• Builds a hierarchy of clusters (agglomerative or divisive) • Represented as a dendrogram • Does not require the number of clusters in advance • Example: Agglomerative Clustering, Divisive Clustering
Density-Based Methods	• Forms clusters based on dense regions of data points • Can discover clusters of arbitrary shape • Can identify outliers as noise • Example: DBSCAN, OPTICS

5 Partitioning Methods

5.1 K-Means Clustering: A Centroid-Based Technique

- **Type:** Centroid-based clustering algorithm.
- **Centroid:** Each cluster is represented by the mean of its points.
- **Requirement:** Number of clusters K must be predefined.
- **Why we need K-Means:**
  - To partition large datasets into meaningful groups automatically.
  - To simplify complex data by reducing it to cluster representatives.
  - Helps identify underlying structure in unlabelled data.

- **Purpose:**
  - To group similar data points together based on distance metrics.
  - To support decision-making by analyzing cluster patterns.
- **Usefulness:**
  - Data compression and summarization.
  - Market segmentation, customer profiling.
  - Image segmentation and pattern recognition.
  - Anomaly or outlier detection.
  - Provides a foundation for other machine learning tasks.

K-Means Algorithm Steps

1. Select K initial centroids randomly from the data points.
2. Assign each data point to the nearest centroid.
3. Recompute the centroids as the mean of all points assigned to each cluster.
4. Reassign each data point to the nearest (updated) centroid.
5. Repeat steps 3 and 4 until centroids do not change significantly (convergence).
6. Output the final clusters and their centroids.

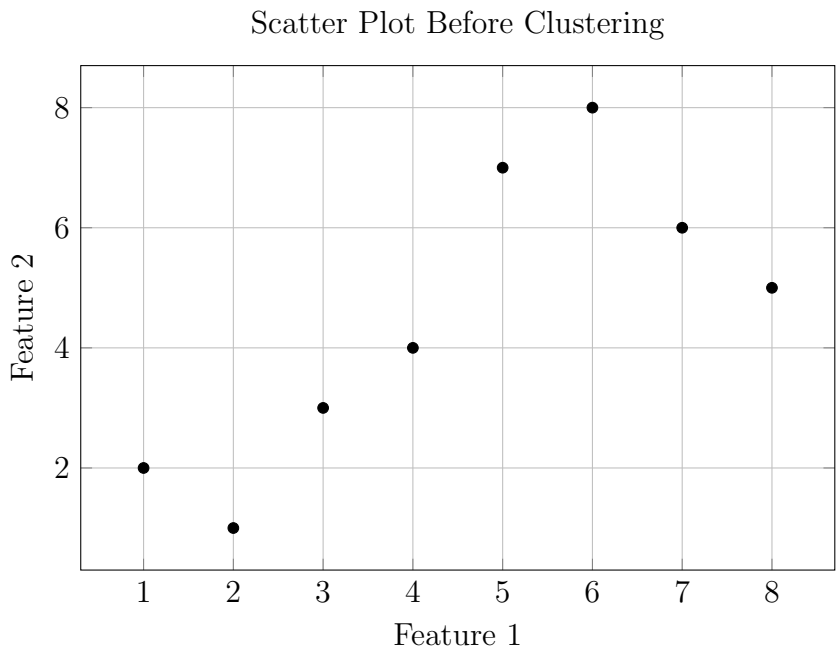
Note: Steps 2 to 5 form the iterative loop of the K-Means algorithm.

5.2 Strengths and Weaknesses of K-Means

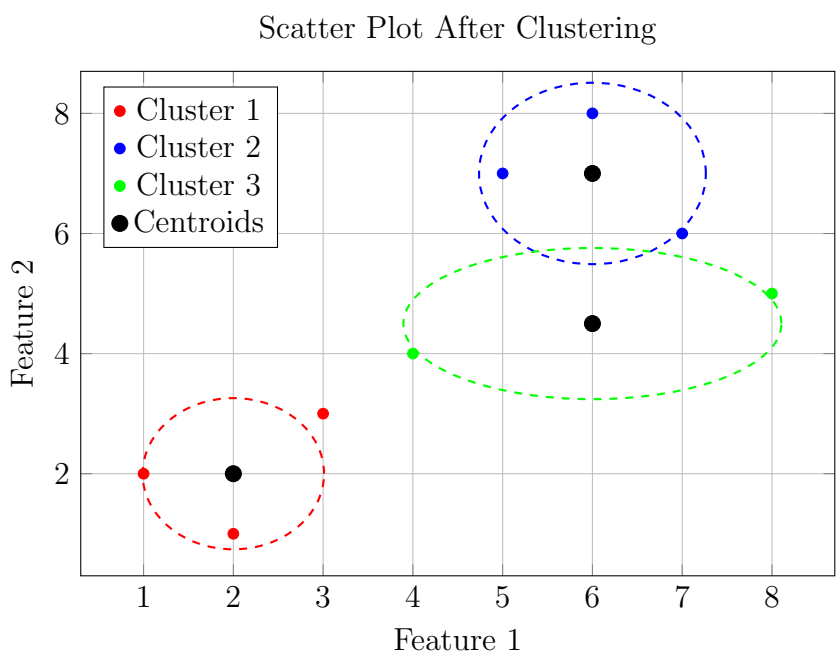
Aspect	Details
Strengths	• Simple and easy to implement. • Fast and efficient for large datasets. • Works well when clusters are spherical and equally sized. • Provides clear cluster assignments. • Easily interpretable results.
Weaknesses	• Requires the number of clusters K to be specified in advance. • Sensitive to initial centroids. • Sensitive to outliers and noise. • Cannot detect non-spherical clusters effectively. • May converge to a local minimum instead of global optimum.

5.3 Scatter Plot Before and After Clustering

Before clustering, data points are unlabelled and scattered:



After applying K-Means, points are grouped into clusters with centroids:



Observation: After clustering, points within each cluster are more similar to each other (high intra-cluster similarity) and dissimilar to points in other clusters (high inter-cluster separation). The black points represent the centroids calculated by the K-Means algorithm, around which each cluster is formed.

5.4 Drawbacks of K-Means and Remedies

- Sensitive to initial centroids – **Use K-Means++ initialization.**

- Requires predefined K – **Use methods like the Elbow Method to choose K.**
- Cannot handle non-spherical clusters – **Use Density-Based clustering like DBSCAN.**
- Sensitive to outliers – **Preprocess data**, remove outliers or use robust clustering methods.

5.5 Elbow Method

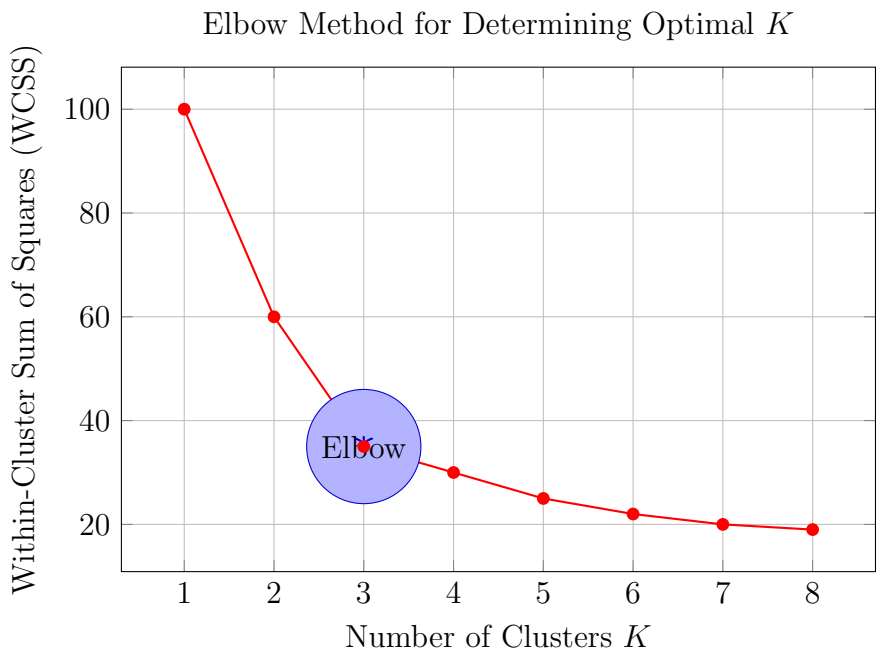
- Determines the **optimal number of clusters** (K).
- Measures homogeneity / heterogeneity within clusters for different K values.
- As K increases:
  - Homogeneity increases (points in clusters are more similar).
  - Heterogeneity decreases (clusters are tighter).
- Beyond a certain K, further increase gives diminishing returns.
- The **elbow point** indicates the optimal K, producing the best clustering performance.

Choosing Initial Centroids

- Proper selection of **initial centroids** is crucial for K-Means clustering.
- Random selection may lead to higher SSE and sub-optimal clusters.
- Repeated runs may not always find optimal clusters due to data spread.
- **Better approach:**
  - Apply hierarchical clustering on a sample to get K clusters.
  - Use the centroids of these clusters as **initial centroids**.
- Useful for small datasets or when K is small.
- Advanced methods like bisecting K-Means and post-processing refinements improve centroid selection and reduce SSE.

Need: To determine the optimal number of clusters K automatically rather than arbitrarily.
Purpose: To find a balance between the number of clusters and the within-cluster homogeneity.
Usefulness: Prevents underfitting (too few clusters) and overfitting (too many clusters) by analyzing cluster compactness.

5.6 Graphical Representation of the Elbow Method



Observation:
– The elbow at $K = 3$ indicates the point beyond which adding more clusters does not significantly reduce WCSS.
– Ensures **high within-cluster homogeneity** and appropriate number of clusters.
– This is the optimal K for K-Means clustering in this dataset.

Advantages

- Simple and computationally efficient
- Suitable for large datasets

Limitations

- Sensitive to outliers
- Requires predefined K
- May converge to local optimum

5.7 Iterative Steps and SSE

- K-Means iteratively **recomputes centroids** to minimize distances within clusters.
- **Euclidean distance** is used to measure the proximity of data points to centroids:

$$\text{dist}(x, c) = \sqrt{\sum_{i=1}^{n} (x_i - c_i)^2}$$

- **Quality of clustering** is measured using **Sum of Squared Errors (SSE)**:

$$SSE = \sum_{k=1}^{K} \sum_{x \in C_k} \text{dist}(x, c_k)^2$$

- Lower SSE indicates better centroid positioning.
- Centroid minimizing SSE is the **mean of cluster points**.
- Limitation: SSE is sensitive to outliers.
- K-Means outcome depends on:
  - Initial random selection of centroids.
  - Number of clusters K.
- To improve reliability: run K-Means multiple times with different initial centroids or vary K.
- Clustering is often used **before classification** to identify subgroups in unlabeled data.

Illustration of Iterations

Figure : Initial Centroids with Data Points

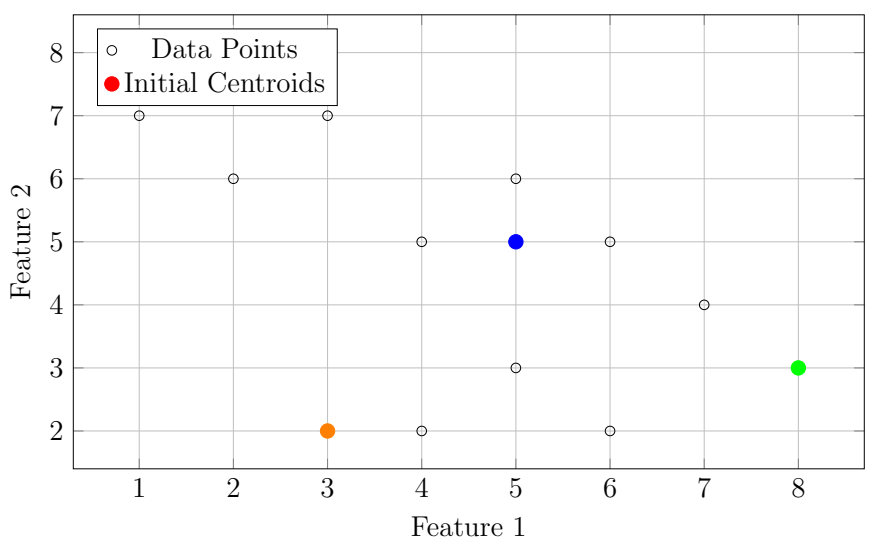
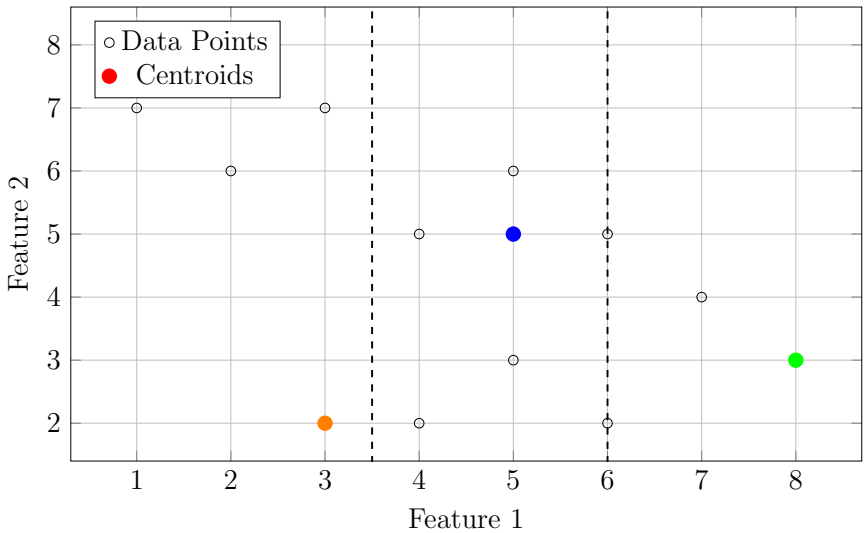
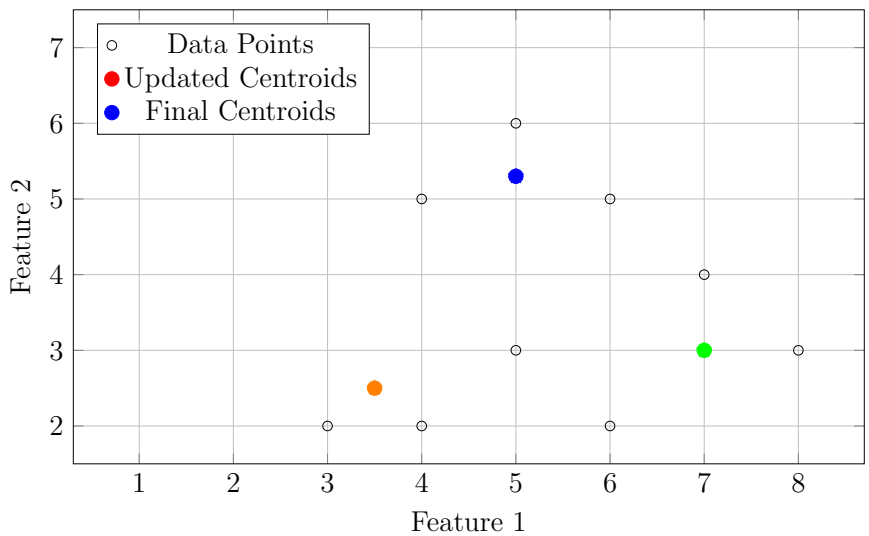


Figure : Iteration 1 – Cluster Assignment with Voronoi-like Boundaries



Figures: Iteration 2 & Final Clusters



Observations:

- Clusters are refined iteratively by assigning points to the nearest centroid and recomputing centroids.
- SSE is minimized at each step.
- Voronoi boundaries help visualize the cluster regions.
- Final clusters depend on initial centroid selection.
- Multiple runs with different centroids can improve global clustering results.

Example Application: Software Defect Clustering

- Clustering identifies subgroups of defects (e.g., GUI, business logic, missing requirements, database-related). - Classification algorithms can then assign new defects to these subgroups for easy developer allocation. - Helps manage unlabeled datasets and discover patterns in software defects.

5.8 K-Medoids: A Representative Object-Based Technique

- **Problem with K-Means:** Sensitive to outliers; means can shift clusters.
- **Example:** 1-D data points: $\{1, 2, 3, 5, 9, 10, 11, 25\}$, K=2.
  - K-Means may group point 9 with $\{1, 2, 3, 5\}$ due to outlier 25.
  - **Skewed clusters** arise because SSE is minimized using mean.
- **K-Medoids Solution:**
  - Uses actual data points as cluster representatives (medoids).
  - Assign points to nearest medoid instead of centroid.
  - SSE is calculated as:
    $$SSE = \sum_{x \in C} dist(x, o)^2$$
    where o is the medoid of cluster C.
  - Less influenced by outliers.
- Uses actual data points as cluster centers (medoids).
- Less sensitive to outliers than K-Means.

K-Means	**K-Medoids**
Centroid = Mean	Medoid = Actual data point
Sensitive to outliers	Robust to outliers
Fast	Computationally expensive

Partitioning Around Medoids (PAM) Algorithm

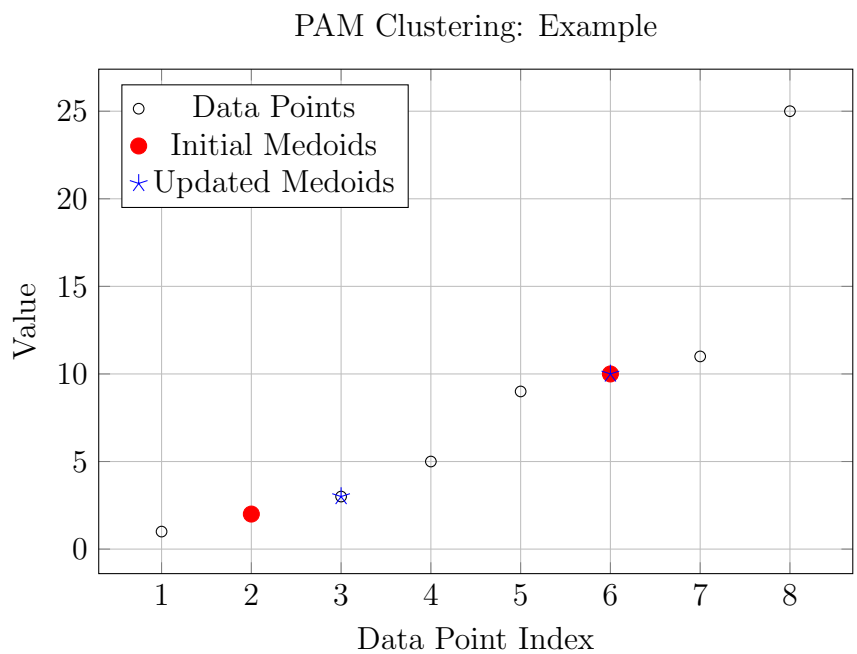
1. Randomly select k points as initial medoids.
2. Assign each remaining point to the cluster of the nearest medoid.
3. For each cluster, randomly select a non-medoid point o'.
4. Swap o' with current medoid o and compute new SSE.
5. If $SSE_{new} < SSE_{old}$, keep the swap and reassign points to nearest medoid.

6. Repeat until no improvement in SSE.

Key Points:

- Iteratively replace medoids to improve cluster quality.
- Each replacement recomputes SSE to check if clustering improves.
- PAM is effective for small datasets but computationally expensive for large n.
- Complexity per iteration: $O(k(n-k)^2)$, more expensive than K-Means for large datasets.
- **Advantage:** Robust to outliers and noise.
- **Limitation:** Not scalable; requires sample-based approaches for large datasets.

Illustration



Observation: After each medoid swap, points are reassigned to the nearest medoid, SSE is recalculated, and clusters are refined iteratively.

5.9 Hierarchical Clustering

Hierarchical clustering is used to group data into a tree-like structure (hierarchy) rather than fixed clusters. It is useful when data has natural nested groups.

Applications

- Organizing employees into departments and roles.
- Discovering hierarchical structures in unlabeled data.

Types of Hierarchical Clustering

- Agglomerative (Bottom-Up): Each object starts as a cluster; clusters are merged iteratively.
- Divisive (Top-Down): Start with one cluster of all objects; iteratively split to form smaller clusters.

Distance Measures Between Clusters

Let C_i and C_j be two clusters, $p_i \in C_i$, $p_j \in C_j$, n_i, n_j number of points in clusters, and m_i, m_j their means:

$$\text{Minimum distance: } D_{\min}(C_i, C_j) = \min_{p_i \in C_i, p_j \in C_j} |p_i - p_j|$$

$$\text{Maximum distance: } D_{\max}(C_i, C_j) = \max_{p_i \in C_i, p_j \in C_j} |p_i - p_j|$$

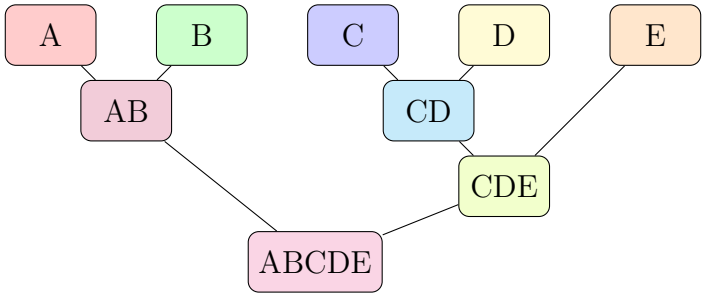
$$\text{Mean distance: } D_{\text{mean}}(C_i, C_j) = |m_i - m_j|$$

$$\text{Average distance: } D_{\text{avg}}(C_i, C_j) = \frac{1}{n_i n_j} \sum_{p_i \in C_i, p_j \in C_j} |p_i - p_j|$$

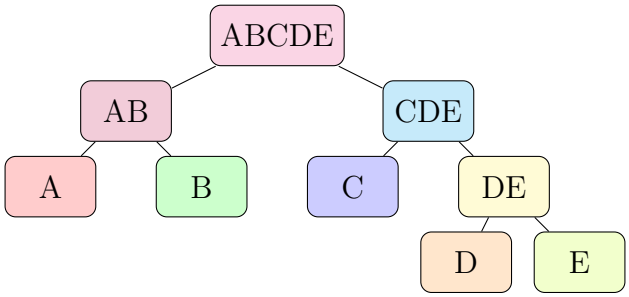
Notes:

- Single linkage uses $D_{\min}$; sensitive to chaining/outliers.
- Complete linkage uses $D_{\max}$; sensitive to outliers.
- Average/mean distance provide more stable clustering.

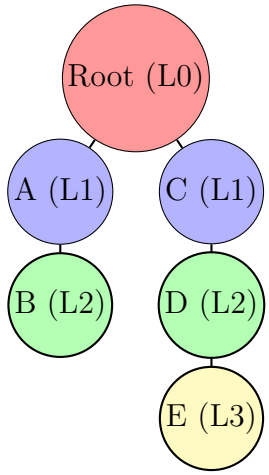
Agglomerative Clustering Steps



Divisive Clustering Steps



Dendrogram Representation



Key Points

- Agglomerative is bottom-up, divisive is top-down.
- No swaps allowed once clusters are merged/split.
- Termination based on threshold distance or number of clusters.
- Dendrograms visualize hierarchical structure and merging/splitting steps.

Distance-based Decisions

- Nearest neighbour / single linkage: merge based on D_{min}.
- Furthest neighbour / complete linkage: merge based on D_{max}.
- Mean / average linkage: merge using D_{mean} or D_{avg} for stability and less sensitivity to outliers.

6 Density-Based Methods - DBSCAN

When using partitioning and hierarchical clustering methods, the resulting clusters are usually spherical. However, these methods fail to capture arbitrarily shaped clusters, such as S-shaped or uneven clusters.

The **density-based clustering approach** addresses this issue by identifying dense and sparse regions in the dataset. DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a popular density-based algorithm that forms clusters by connecting regions with high density.

7 Finding Patterns Using Association Rules

Association rule mining identifies interesting relationships hidden in large datasets. This method is also called **association analysis**, where relationships are represented as rules comprising frequent itemsets. A common application is the **Market Basket Analysis**, used by retailers to identify items frequently bought together for cross-selling opportunities.

For example, if customers often buy *Bread* and *Milk* together with *Egg*, it is beneficial for retailers to place these items nearby or create combo offers.

7.1 Example Association Rule

$$\{Bread, Milk\} \rightarrow \{Egg\}$$

7.2 Applications

Association analysis is widely used in:

- Market Basket Analysis
- Bioinformatics
- Medical diagnosis
- Scientific data analysis
- Web data mining

8 Common Terminologies

8.1 Itemset

An **itemset** is a group of one or more items that appear together in transactions. For example, $\{Bread, Milk, Egg\}$ is a 3-itemset.

Mathematically, if $I = \{i_1, i_2, \ldots, i_m\}$ are items and $T = \{t_1, t_2, \ldots, t_n\}$ are transactions, then each transaction t contains a subset of I. A **k-itemset** contains k items.

8.2 Support Count

The **support count** of an itemset is the number of transactions in which it appears:

$$\text{Support Count}(X) = |\{t \in T \mid X \subseteq t\}|$$

where $|\cdot|$ denotes the cardinality of the set.

Example: In Table 1, $\{Bread, Milk, Egg\}$ occurs 3 times.

8.3 Association Rule

An **association rule** is an expression $X \to Y$, where X and Y are disjoint itemsets ($X \cap Y = \emptyset$). It indicates that if X is purchased, Y is likely to be purchased.

8.3.1 Support and Confidence

- **Support:** Proportion of transactions containing both X and Y.

$$\text{Support}(X \to Y) = \frac{\text{Support Count}(X \cup Y)}{N}$$

- **Confidence:** Conditional probability that Y is purchased given X is purchased.

$$\text{Confidence}(X \to Y) = \frac{\text{Support Count}(X \cup Y)}{\text{Support Count}(X)}$$

Example: For the rule $\{Bread, Milk\} \to \{Egg\}$, the confidence is 0.75.

9 Market Basket Transaction Data

Transaction Number	Purchased Items
1	{Bread, Milk, Egg, Butter, Salt, Apple}
2	{Bread, Milk, Egg, Apple}
3	{Bread, Milk, Butter, Apple}
4	{Milk, Egg, Butter, Apple}
5	{Bread, Egg, Salt}
6	{Bread, Milk, Egg, Apple}

Table 1: Simplified Market Basket Transaction Data

10 Support and Confidence Calculation

Let the total number of transactions be

$$N = 6$$

10.1 Support of Individual Items

The support of an item X is defined as

$$\text{Support}(X) = \frac{\text{Number of transactions containing } X}{N}$$

Item	Frequency	Support
Bread	5	$\frac{5}{6} = 0.833$
Milk	5	$\frac{5}{6} = 0.833$
Egg	5	$\frac{5}{6} = 0.833$
Butter	3	$\frac{3}{6} = 0.500$
Apple	5	$\frac{5}{6} = 0.833$
Salt	2	$\frac{2}{6} = 0.333$

Table 2: Support of Individual Items

10.2 Support of Itemsets

$$\text{Support}(\{Bread, Milk\}) = \frac{4}{6} = 0.667$$

$$\text{Support}(\{Milk, Egg\}) = \frac{4}{6} = 0.667$$

$$\text{Support}(\{Bread, Milk, Egg\}) = \frac{3}{6} = 0.500$$

10.3 Confidence of Association Rules

The confidence of a rule $X \to Y$ is defined as

$$\text{Confidence}(X \to Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Rule 1: Bread $\to$ Milk

$$\text{Confidence} = \frac{\text{Support}(\{Bread, Milk\})}{\text{Support}(\{Bread\})} = \frac{4/6}{5/6} = \frac{4}{5} = 0.80$$

Rule 2: Milk $\to$ Egg

$$\text{Confidence} = \frac{4/6}{5/6} = \frac{4}{5} = 0.80$$

Rule 3: Bread, Milk $\to$ Egg

$$\text{Confidence} = \frac{3/6}{4/6} = \frac{3}{4} = 0.75$$

Rule 4: Egg $\to$ Apple

$$\text{Confidence} = \frac{4/6}{5/6} = \frac{4}{5} = 0.80$$

10.4 Summary of Association Rules

Rule	Support	Confidence
Bread → Milk	0.667	0.80
Milk → Egg	0.667	0.80
Bread, Milk → Egg	0.500	0.75
Egg → Apple	0.667	0.80

Table 3: Support and Confidence of Association Rules

11 Notes on Analysis

- Association rules are unsupervised; no prior labeling is needed.
- Low support indicates a rule may have occurred by chance.
- High confidence indicates the reliability of the rule.
- Useful for discovering patterns in large datasets where manual analysis is infeasible.

12 The Apriori Algorithm for Association Rule Learning

12.1 Challenges in Association Rule Mining

The primary challenge in association rule learning arises from the large volume of transactional data and the exponential growth of possible itemsets. If a dataset contains k distinct items, then the total number of possible itemsets is

$$2^k$$

including the null itemset.

For example, if a seller deals with 100 distinct items, then the total number of possible itemsets is 2^{100}, which is computationally infeasible to evaluate exhaustively. Therefore, it is necessary to prune unimportant itemsets and focus only on those that are meaningful and manageable.

12.2 Support and Confidence Thresholds

To reduce the search space, two thresholds are defined:

- **Minimum Support (minS)**

- **Minimum Confidence (minC)**

Given a set of transactions T, the objective is to discover all association rules whose support is at least minS and confidence is at least minC. Even with moderate values such as minS = 20% and minC = 50%, a large percentage of rules are discarded. This observation motivates early pruning of itemsets to avoid unnecessary computations.

12.3 Decoupling Support and Confidence

The support of an association rule $X \to Y$ depends only on the support of the itemset $X \cup Y$. Therefore, multiple rules generated from the same itemset share the same support.

For example, the following rules all have identical support because they are derived from the same itemset $\{Bread, Milk, Egg\}$:

$$\begin{aligned}
\{Bread, Milk\} &\to \{Egg\} \\
\{Bread, Egg\} &\to \{Milk\} \\
\{Milk, Egg\} &\to \{Bread\} \\
\{Bread\} &\to \{Milk, Egg\} \\
\{Milk\} &\to \{Bread, Egg\} \\
\{Egg\} &\to \{Bread, Milk\}
\end{aligned}$$

If the itemset $\{Bread, Milk, Egg\}$ is infrequent, then all these rules can be discarded without computing their confidence values.

12.4 Key Steps in Association Rule Mining

1. **Frequent Itemset Generation:** Identify all itemsets whose support is greater than or equal to minS. These are called *frequent itemsets*.

2. **Rule Generation:** Generate association rules from the frequent itemsets and retain only those rules whose confidence is greater than or equal to minC. These rules are known as *strong rules*.

The computational cost of frequent itemset generation is significantly higher than that of rule generation, motivating the use of optimized algorithms such as Apriori.

12.5 Apriori Principle

The Apriori algorithm is based on a simple but powerful principle:

> *If an itemset is frequent, then all of its subsets must also be frequent.*

Equivalently, if an itemset is infrequent, then all of its supersets must also be infrequent. This property is known as the **anti-monotone property** of the support measure.

12.6 Support-Based Pruning

The support of an itemset never exceeds the support of any of its subsets. Therefore, if an item does not meet the minimum support threshold, then any itemset containing that item can be safely discarded.

For example, if the item *Salt* is found to be infrequent in a market basket dataset, then all itemsets containing Salt can be eliminated from further consideration. This significantly reduces the number of candidate itemsets and improves computational efficiency.

12.7 Computational Complexity

Without pruning, the total number of comparisons required to compute support is given by

$$T \times N \times L$$

where:

- T is the number of transactions,
- N is the number of candidate itemsets,
- L is the maximum transaction length.

This brute-force approach is computationally expensive and impractical for large datasets.

12.8 Iterative Process of Apriori

The Apriori algorithm operates in multiple iterations:

- Iteration 1 evaluates all one-itemsets.
- Iteration 2 evaluates candidate two-itemsets generated from frequent one-itemsets.
- Iteration k evaluates candidate k-itemsets generated from frequent $(k-1)$-itemsets.

Candidate itemsets are pruned before evaluation if any of their subsets are infrequent.

For example, if $\{X\}$, $\{Y\}$, and $\{Z\}$ are frequent but $\{W\}$ is not, then only $\{X, Y\}$, $\{X, Z\}$, and $\{Y, Z\}$ are considered in the next iteration.

If $\{X, Z\}$ is found to be infrequent, then $\{X, Y, Z\}$ is not generated in the subsequent iteration.

12.9 Rule Generation Phase

Once all frequent itemsets have been identified, association rules are generated from all non-empty subsets. For example, from the frequent itemset $\{X, Y\}$, the following rules can be formed:

$$X \to Y, \quad Y \to X$$

Each rule is evaluated against the minimum confidence threshold, and only those satisfying the threshold are retained as strong rules.

12.10 Strengths and Limitations of Apriori

Strengths	**Limitations**
Simple and intuitive algorithm	Requires multiple scans of the database
Effective pruning of the search space using support	Performance degrades for very large or dense datasets
Widely used in market basket analysis	Generates a large number of candidate itemsets in early iterations

Table 4: Strengths and Limitations of the Apriori Algorithm

Chapter 10: BASICS OF NEURAL NETWORK

February 18, 2026

10.1 EVOLUTION FROM MACHINE LEARNING TO NEURAL NETWORKS

In the previous chapters of Machine Learning, we studied:-**Supervised Learning (Classification & Regression), Unsupervised Learning (Clustering & Association Rules), Feature Engineering, Model Evaluation.** These methods rely heavily on statistical formulas and predefined rules. However, many real-world problems are:

- Highly complex
- Non-linear
- Pattern-intensive

Traditional algorithms sometimes fail to capture such complexity. This limitation led to the development of Neural Networks, which are designed to learn patterns automatically instead of depending entirely on explicit programming.

10.1.1 What is a Neural Network?

A **Neural Network** is a computational model inspired by the human brain. A system made of interconnected units called **neurons** which is designed to recognize patterns, relationships, and structures in data. It works by receiving inputs, processing them through multiple layers and producing an output.
The main idea is learning by adjusting internal connections, similar to how humans learn from experience.

10.1.2 Artificial Neural Network (ANN)

An Artificial Neural Network (ANN) is a digital or electronic simulation of a biological neural system, composed of artificial neurons and organized into layers:

- Input Layer
- Hidden Layer
- Output Layer

Each artificial neuron performs small calculations, and when thousands or millions of such neurons work together, they can solve complex tasks.

10.1.3 Why Neural Networks Are Important?

Neural Networks are powerful because they can:

- Handle large volumes of data
- Capture non-linear relationships
- Automatically extract features
- Improve performance with experience

Applications:

- Image Recognition (Face detection, object detection)
- Speech Recognition (Voice assistants)
- Natural Language Processing (Chatbots, translation)
- Medical Diagnosis
- Stock Market Prediction
- Self-Driving Cars
- Recommendation Systems

Human Learning vs Machine Learning vs Neural Networks

Aspect	Human Learning	Traditional ML	Neural Networks
Basis	Experience and brain	Statistics and rules	Brain-inspired structure
Adaptability	Very high	Moderate	Very high
Pattern Detection	Natural	Limited	Strong
Parallel Processing	Yes	No	Yes

10.1.4 Key Characteristics of Neural Networks

1. **Parallelism** – Multiple neurons work simultaneously.
2. **Adaptability** – Learns by adjusting weights.
3. **Non-Linearity** – Can model complex relationships.
4. **Generalization** – Can predict unseen data.
5. **Fault Tolerance** – Even if some neurons fail, the system still works.

10.2 UNDERSTANDING THE BIOLOGICAL NEURON

To understand ANN, we must first understand the Biological Neuron, since ANN is inspired by it.

10.2.1 What is a Biological Neuron?

A Neuron (also called a nerve cell) is:

- The fundamental unit of the human nervous system.
- Responsible for receiving, processing, and transmitting information.
- The human brain contains billions of neurons interconnected in complex networks.

These networks enable humans to think, learn, remember, make decisions and perform motor functions.

Structure of a Biological Neuron

A biological neuron has four major components:

(a) Dendrites

- Branch-like extensions.
- Receive signals from other neurons.
- Act as input channels.
- The more dendrites, the more signals a neuron can receive.

(b) Cell Body (Soma)

- Central part of the neuron.
- Processes incoming signals.
- Decides whether the neuron should activate or not.
- Contains the nucleus and essential cellular machinery.

(c) Axon

- Long, tail-like structure.
- Carries signals away from the cell body.
- Acts as the output channel.
- Can be extremely long compared to other parts.

(d) Synapse

- Junction between two neurons.
- The point where signals are transmitted.
- The strength of a synapse determines how strongly neurons are connected.
- Synaptic strength changes with learning and experience.

How Signal Transmission Occurs

The process of communication between neurons follows these steps:

1. **Signal Reception** – Dendrites receive signals from nearby neurons.
2. **Signal Integration** – Cell body combines all incoming signals.
3. **Threshold Check** – If combined signal exceeds a certain threshold, the neuron fires.
4. **Signal Transmission** – Axon sends the signal to the next neuron via synapse.

This process happens millions of times per second in the human brain.

0.1 10.2.2 Key Functional Characteristics

(i) Threshold Activation

- A neuron fires only when input strength crosses a limit.
- Prevents unnecessary activation.

(ii) Parallel Processing

- Multiple neurons work simultaneously.
- Enables fast decision making.

(iii) Plasticity (Learning Ability)

- Synaptic strengths change over time.
- Learning occurs by strengthening or weakening connections.

10.3 THE ARTIFICIAL NEURON

The artificial neuron simulates a biological neuron digitally.
Components:

1. **Inputs:** $x_1, x_2, \ldots, x_n$
2. **Weights:** $\omega_1, \omega_2, \ldots, \omega_n$ (measure connection strength or importance)
3. **Summation Function:** $y_{sum} = \sum_i \omega_i x_i$. Sometimes includes a bias 'b' $\rightarrow y_{sum} = b + \sum_i \omega_i x_i$. Weights can be

   - Positive $\rightarrow$ Excitatory (increase output influence)
   - Negative $\rightarrow$ Inhibitory (decrease output influence)

   Real-Life Analogy: Imagine deciding your exam result, with 'Homework weight' = 20%, 'Midterm weight'= 30%, 'Final exam weight'= 50%. Here, weights determine which factor influences the final decision more.
   Bias(b):

- Bias shifts the activation threshold.
- It is similar to intercept in linear regression.

Real-Life Analogy: Bias is like a grace mark in exams that shifts your total score upward.

4. **Activation Function $f(z)$:**

   - Applies a non-linear transformation to the weighted sum.
   - Determines whether neuron should "fire" or stay inactive.
   - Without activation function, the network becomes purely linear, limiting learning ability.

   Output Formula $y_{sum} = f(y_{sum})$

Example for working Process of Artificial Neuron:(Loan Approval System)- Inputs: Salary, Credit Score, Age. Weights determine influence; if threshold crossed $\rightarrow$ Loan Approved.

10.4 TYPES OF ACTIVATION FUNCTIONS

Activation functions introduce **non-linearity** enabling neural networks to model complex real-world patterns.

10.4.1 Identity Function

$$f(x) = x$$

- Linear function.
- Mainly used in input layers.
- Cannot model complex patterns.

Analogy: Copying homework without understanding.

10.4.2 Threshold / Step Function

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

- Produces binary output.
- Used in early perceptrons.

10.4.3 ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- Widely used in deep learning.
- Fast computation.
- Helps prevent vanishing gradient problem.

Analogy: Like a one-way valve allowing only positive flow.
Limitation: "Dead neuron" problem if values stay negative.

10.4.4 Sigmoid Function

There are two types of sigmoid function:

1. **Binary sigmoid function:**

$$f(x) = \frac{1}{1 + e^{-kx}}$$

   - Output range: $(0, 1)$
   - Useful in binary classification.
   - Interpretable as probability.

   Example: Spam detection probability.

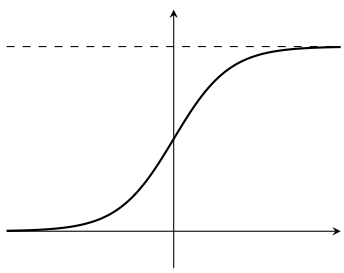
2. **Bipolar sigmoid function:**

$$f(x) = \frac{1 - e^{-kx}}{1 + e^{-kx}}$$

   - Output range: $(-1, 1)$
   - Outputs have two polarities: negative and positive.
   - Because it is zero-centered, it often performs better than the standard sigmoid in some learning tasks.

   Example: Temperature Scale Analogy:
   Think of a thermometer ranging from –1 (very cold) to +1 (very hot), with 0 being neutral. Small changes near 0 are noticeable, but changes at extremes (very hot or very cold) make little difference — similar to the flattening ends of the curve.

(a) Binary Sigmoid Function



(b) Bipolar Sigmoid Function

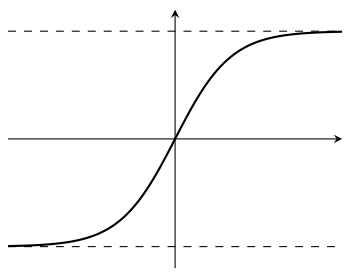


Fig. 10.1 Sigmoid Function

10.4.5 Hyperbolic Tangent (tanh)

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Output range: $(-1, 1)$
- Zero-centered.
- Faster convergence than sigmoid.

Analogy: Temperature scale from cold to hot.

10.5 EARLY IMPLEMENTATIONS OF ANN

These early models laid the foundation for modern neural networks.

10.5.1 McCulloch–Pitts Model of Neuron (1943)

- Binary inputs and outputs.
- No learning capability.
- Uses threshold logic.

$$y = \begin{cases} 1 & \sum w_i x_i \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Example: Logical AND/OR gates.

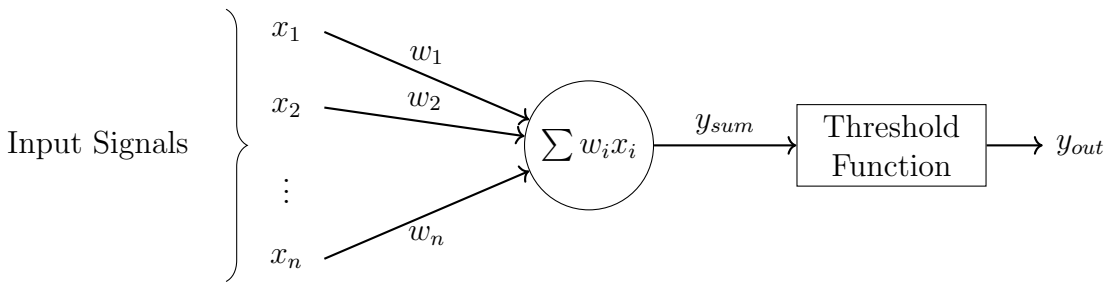


Fig. 10.2 McCulloch–Pitts Neuron

10.5.2 Rosenblatt's Perceptron (1958)

- First learning neural model.
- Adjusts weights using error correction.
- Works only for linearly separable data.

Mathematical Rule

$$w_{\text{new}} = w_{\text{old}} + \eta(t - y)x$$

Where η =learning rate. **Limitation:** Cannot solve XOR problem. **Analogy:** Learning from mistakes in exams.

10.5.3 ADALINE (Adaptive Linear Neuron)

Developed by Bernard Widrow.

- Uses Least Mean Square (LMS) rule.
- Produces continuous output.

$$y = \sum w_i x_i + b$$

Example: Noise-cancelling headphones adapting sound waves.

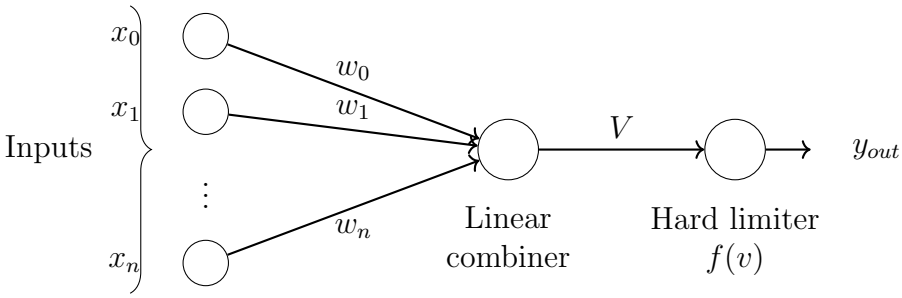


Fig. 10.3 Rosenblatt's Perceptron

Example;
Noise cancellation headphones adjusting sound waves adaptively.

Importance of Early Models

- Introduced weighted connections
- Learning algorithms
- Threshold logic
- Adaptive computation

These concepts evolved into Multi-Layer Perceptrons and Deep Neural Networks used in:

- Face recognition
- Medical imaging
- Autonomous vehicles
- Chatbots

Central Insight: Early ANN models transformed machines from **rule-followers** into **pattern-learners**.

10.6 ARCHITECTURES OF NEURAL NETWORK

Neural network architecture defines how neurons are arranged and connected. The architecture determines how signals flow and how learning occurs.
There are multiple possibilities for connecting the neurons based on which architecture we are going to adopt for a specific solution. Some of the choices are listed below:

- There may be just two layers of neuron in the network – the input and output layer.
- Other than the input and output layers, there may be one or more intermediate 'hidden' layers of neuron.
- The neurons may be connected with one or more of the neurons in the next layer.
- The neurons may be connected with all neurons in the next layer.
- There may be single or multiple output signals. If there are multiple output signals, they might be connected with each other.
- The output from one layer may become input to neurons in the same or preceding layer.

10.6.1 Single-Layer Feed Forward Network

- Simplest ANN structure.
- Contains input layer and output layer.
- No hidden layer.
- Signals flow only in one direction (input $\rightarrow$ output).
- Computation occurs only in the output layer.
- Suitable for simple, linearly separable problems.

The structure of this network is depicted in Figure 10.17.

The input layer consists of a set of m input neurons $X_1, X_2, \ldots, X_m$ connected to each of the n output neurons $Y_1, Y_2, \ldots, Y_n$. The connections carry weights $w_{11}, w_{12}, \ldots, w_{mn}$.

The net signal input to the output neurons is given by

$$y_{in_k} = x_1 w_{1k} + x_2 w_{2k} + \ldots + x_m w_{mk} = \sum_{i=1}^{m} x_i w_{ik}$$

for the k-th output neuron. The signal output from each output neuron will depend on the activation function used.

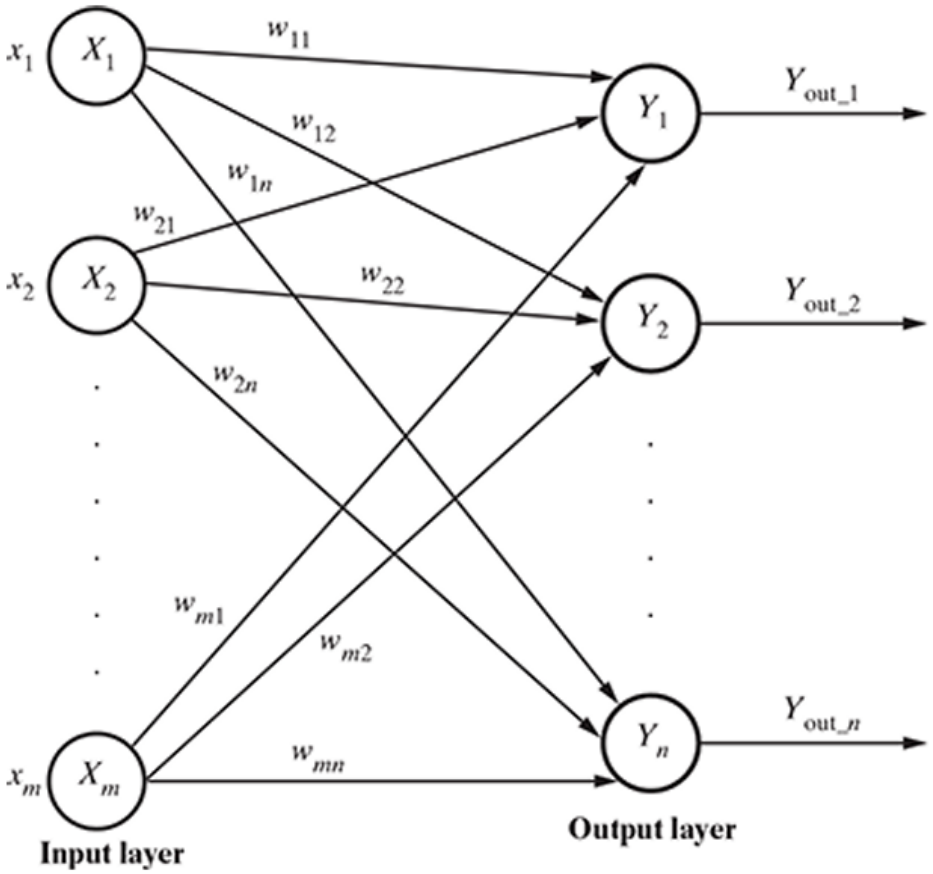


FIG. 10.17 Single-layer feed forward

10.6.2 Multi-Layer Feed Forward Network

- Contains one or more hidden layers between input and output.
- Signals flow forward only.
- Can model complex, non-linear relationships.
- Each layer may have different numbers of neurons.
- Commonly used in practical applications.

The structure of this network is depicted in Figure 10.18.

For example, the one shown in Figure 10.18 has m neurons in the input layer and r neurons in the output layer, and there is only one hidden layer with n neurons.

The net signal input to the neuron in the hidden layer is given by

$$y_{in_k} = x_1 w_{1k} + x_2 w_{2k} + \ldots + x_m w_{mk} = \sum_{i=1}^{m} x_i w_{ik}$$

for the k-th hidden layer neuron. The net signal input to the neuron in the output layer is given by

$$z_{in_k} = y_{out_1} w'_{1k} + y_{out_2} w'_{2k} + \ldots + y_{out_n} w'_{nk} = \sum_{i=1}^{n} y_{out_i} w'_{nk}$$

for the k-th output layer neuron.

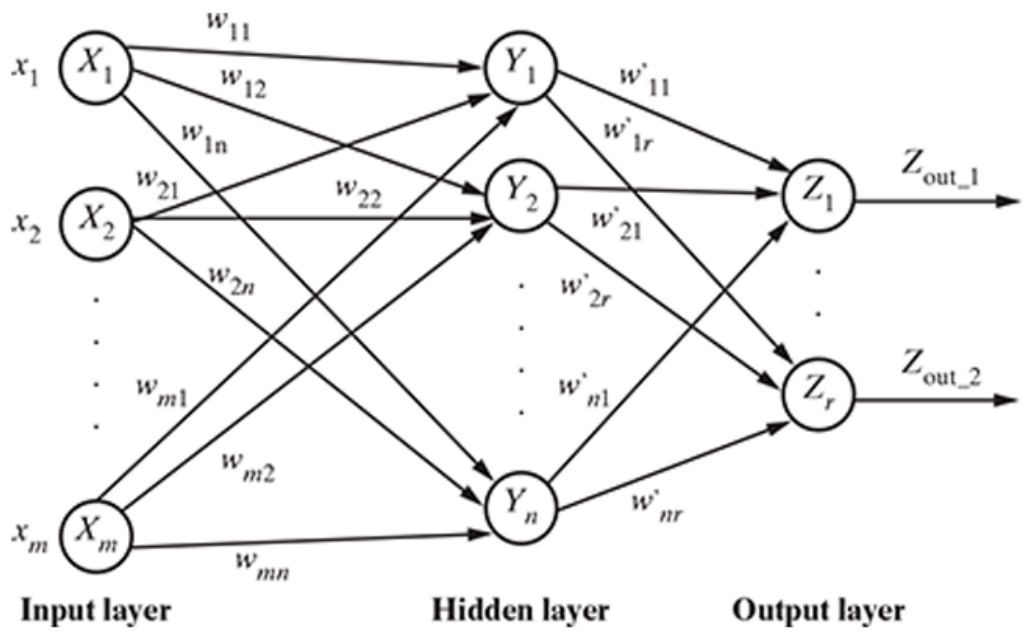


FIG. 10.18 Multi-layer feed forward

10.6.3 Competitive network

- Similar to single-layer feed forward network.
- Output neurons are connected with each other.
- Output neurons compete to represent input.
- Used mainly in unsupervised learning.
- Useful for clustering problems.

The structure of this network is depicted in Figure 10.19.

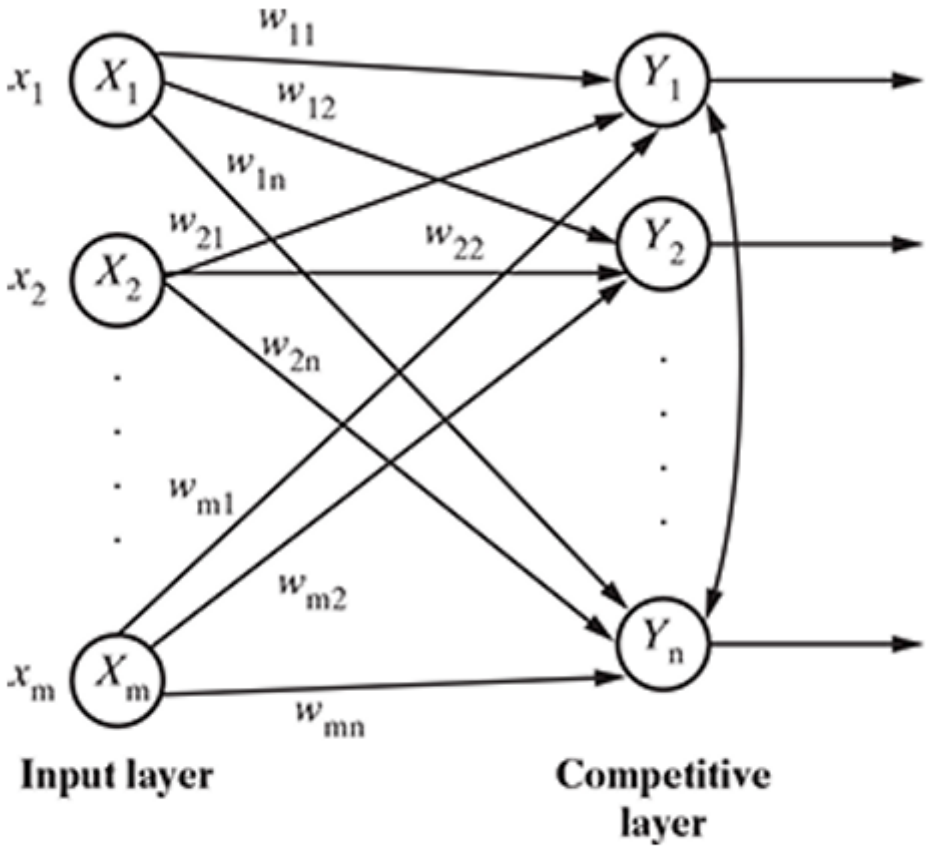


FIG. 10.19 Competitive network

10.6.4 Recurrent network

- Includes feedback connections.
- Output can be fed back to input or same layer.
- May contain self-loops.
- Suitable for sequence and time-dependent data.
- Signals do not strictly move in one direction.

10.7 LEARNING PROCESS IN ANN

Learning in ANN involves determining optimal weights and network structure. There are four major aspects which need to be decided:

1. The number of layers in the network
2. The direction of signal flow
3. The number of nodes in each layer
4. The value of weights attached with each interconnection between neurons

10.7.1 Number of layers

- Single-layer: simple problems.
- Multi-layer: complex problems.
- One hidden layer can approximate many continuous functions.

10.7.2 Direction of signal flow

- Feed forward: input $\rightarrow$ output only.
- Recurrent: includes feedback loops.

10.7.3 Number of nodes in layers

- Input nodes = number of features.
- Output nodes = number of classes/outputs.
- Hidden nodes are user-defined.
- Too many nodes $\rightarrow$ overfitting and high computation cost.

10.7.4 Weight of interconnection between neurons

- Learning means updating connection weights.
- Start with random weights.
- Adjust iteratively to reduce classification error.
- Stopping criteria:
  - Error below threshold
  - Maximum number of iterations reached

10.8 BACKPROPAGATION

The backpropagation algorithm is applicable for multi-layer feed forward networks. It is a supervised learning algorithm which continues adjusting the weights of the connected neurons with an objective to reduce the deviation of the output signal from the target output. This algorithm consists of multiple iterations, also known as epochs. Each epoch consists of two phases-

- A **forward phase** in which the signals flow from the neurons in the input layer to the neurons in the output layer through the hidden layers. The weights of the interconnections and activation functions are used during the flow. In the output layer, the output signals are generated.
- A **backward phase** in which the output signal is compared with the expected value. The computed errors are propagated backwards from the output to the preceding layers. The errors propagated back are used to adjust the interconnection weights between the layers.

The iterations continue till a stopping criterion is reached. Figure 10.21 depicts a reasonably simplified version of the backpropagation algorithm.

One main part of the algorithm is adjusting the interconnection weights. This is done using a technique termed as gradient descent. In simple terms, the algorithm calculates the partial derivative of the activation function by each interconnection weight to identify the 'gradient' or extent of change of the weight required to minimize the cost function. Quite understandably, therefore, the activation function needs to be differentiable. Let us try to understand this in a bit more details.

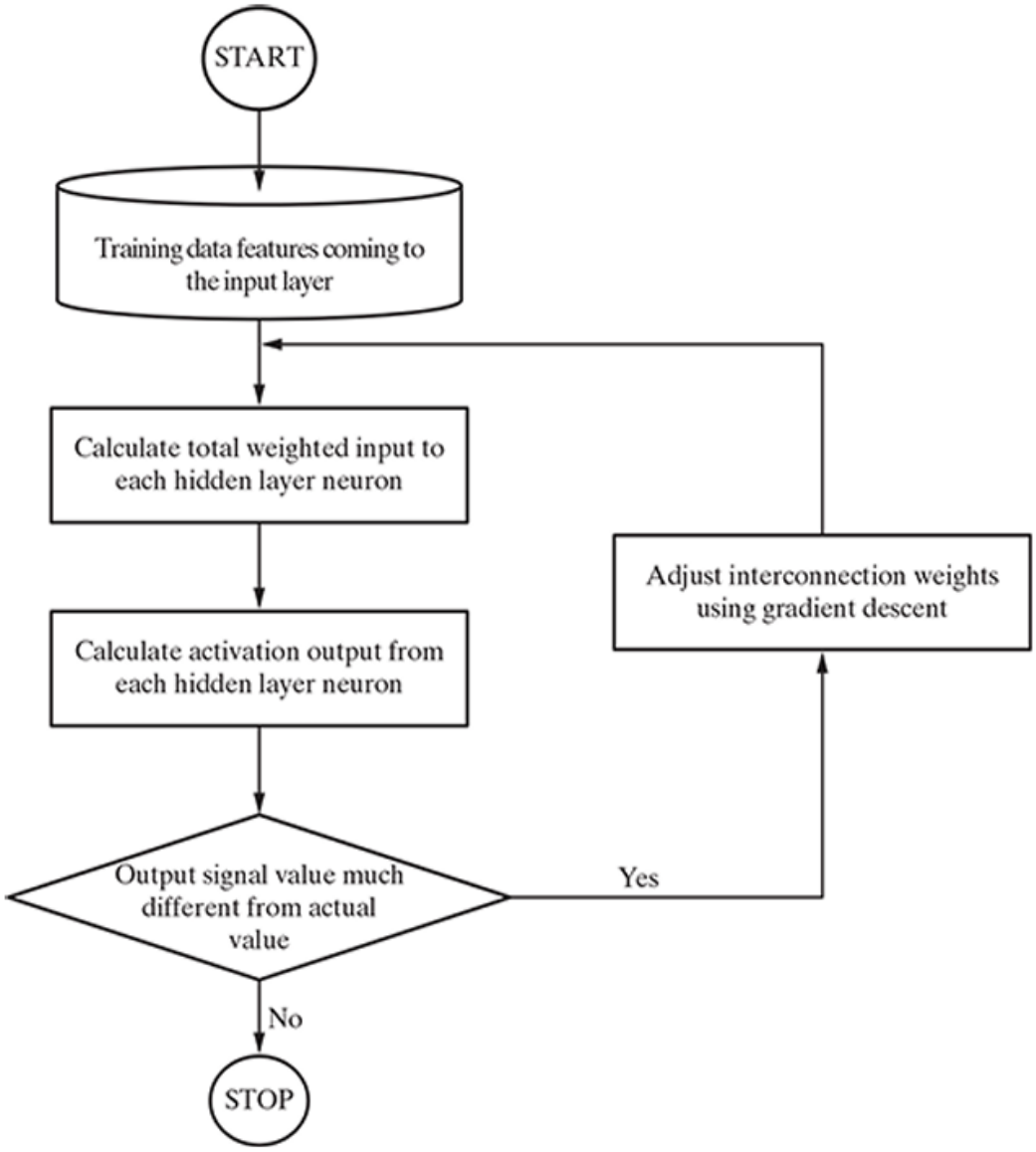


FIG. 10.21 Backpropagation algorithm

10.9 DEEP LEARNING

Deep learning refers to neural networks with multiple hidden layers.

Shallow Neural Network

- Usually 1–2 hidden layers.

Deep Neural Network

- More than 3 hidden layers.
- Handles complex problems.
- Computationally expensive.
- Requires GPU computing for efficient training.

Key Points

- Deep learning is an advanced form of multi-layer neural networks.
- Used in large-scale problems like image recognition.
- Higher depth increases computational complexity.
- Modern deep learning relies heavily on parallel processing (GPU).

Other Types of Learning
Chapter 11

OBJECTIVE OF THE CHAPTER

In the previous chapters, we have got good understanding of supervised learning algorithm and unsupervised learning algorithm. In this chapter, we will discuss learning types which were not covered in the earlier chapters which includes representation learning, active learning, instance-based learning, association rule, and ensemble learning. By the end of this chapter, you will gain sufficient knowledge in all the aspects of learning and become ready to start solving problems on your own.

11.1 INTRODUCTION

In this chapter, we will discuss learning types which were not covered in the earlier chapters which include representation learning, active learning, instance-based learning, association rule, and ensemble learning.

11.2 REPRESENTATION LEARNING

Another name for representation learning is feature learning. While many enterprises nowadays retain substantial amounts of business-related data, most of those are often unstructured and unlabelled. Creating new labels is generally a time-consuming and expensive endeavour. As a result, machine learning algorithms that can straight away mine structures from unlabelled data to improve the business performance are reasonably valued.

Representation learning is one such type where the extraction from the overall unlabelled data happens using a 'neural network'. The main objective of representation learning (feature learning) is to find an appropriate representation of data based on features to perform a machine learning task. Representation learning has become a field in itself because of its popularity.

Example: Classification of Triangle

Characteristics of triangle:

- Corners (vertices)
- Sides
- Degree of angle
- Sum of three angles is equal to 180°

System working:

- Input – Triangular image

- Representation – Three angles, length of sides
- Model – Applies rules as per Table 11.1 to detect the type of triangle

When other shapes like square, rectangle, circle are input, designing such features requires deep domain expertise. Deep Learning learns features on its own by feeding image as input.

Representation learning is widely used in words, speech recognition, signal processing, music, and image identification.

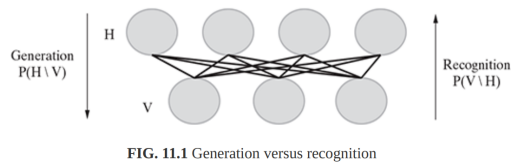


Figure 1: Generation vs Recognition

Generation vs Recognition:

If causes (H) explain the data (V) observed, it is called Recognition. If unknown causes (H) generate the data (V), it is called Generative (Generation).

Representation learning can be:

1. Supervised neural networks and multilayer perceptron
2. Independent component analysis (unsupervised)
3. Autoencoders (unsupervised)
4. Various forms of clustering

11.2.2 Independent Component Analysis (ICA)

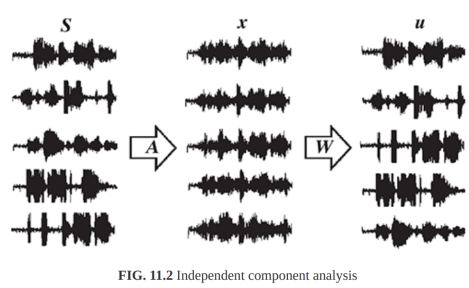


Figure 2: ICA

ICA is similar to PCA, except components are independent and uncorrelated. Used for separating unknown source signals from observed linear mixtures.

- ICA assumes that the observed data X is a linear mixture of statistically independent source signals S.
- The mixing model can be written as:

$$X = AS$$

  where A is an unknown mixing matrix.
- The objective of ICA is to estimate an unmixing matrix W such that:

$$U = WX$$

  where U approximates the original independent sources.
- Unlike PCA, which ensures only uncorrelated components, ICA ensures statistical independence among components.
- Independence is a stronger condition than uncorrelatedness and requires higher-order statistics.
- Maximizing the joint entropy $H(y)$ minimizes mutual information among output components.
- Minimizing mutual information makes the output components as independent as possible.

11.2.3 Autoencoders

Autoencoders belong to the neural network family and are similar to Principal Component Analysis (PCA). They are more flexible than PCA. Autoencoders represent both linear and non-linear transformation, whereas PCA represents only linear transformation.

The neural network's (autoencoder) target output is its input (x) in a different form (x'). In Autoencoders, the dimensionality of the input is equal to the dimensionality of the output, and essentially what we want is:

$$x' = Decode(Encode(x))$$

To get the value from the hidden layer, we multiply the (input to hidden) weights by the input. To get the value from the output, we multiply (hidden to output) weights by the hidden layer values.

$$Z = f(Wx)$$

$$Y = g(Vz)$$

$$Y = g(Vf(Wx)) = VWx$$

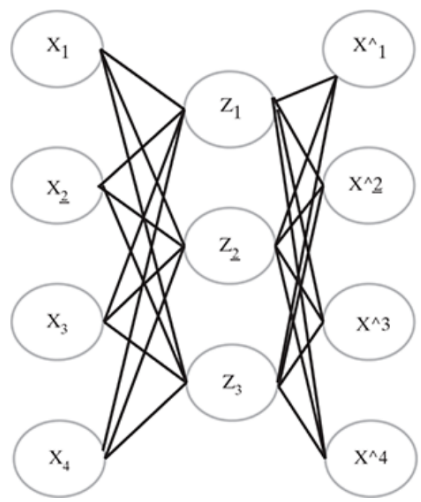


FIG. 11.3 Autoencoders

Figure 3: Autoencoders

Objective Function:

$$J = \sum [X_n - VWx^{(n)}]^2$$

Autoencoder training algorithm can be summarized as below:

1. For each input x, do a feed-forward pass to compute activations at all hidden layers, then at the output layer to obtain an output X'.
2. Measure the deviation X' from the input X.
3. Backpropagate the error through the net and perform weight updates.

11.3 ACTIVE LEARNING

Active learning is a type of semi-supervised learning in which a learning algorithm can interactively query (question) the user to obtain the desired outputs at new data points.

Let P be the population set of all data under consideration.

During each iteration i, P (total population) is broken up into three subsets:

- $P(K, i)$: Data points where the label is known (K).
- $P(U, i)$: Data points where the label is unknown (U).
- $P(C, i)$: A subset of $P(U, i)$ that is chosen (C) to be labelled.

Current research in this type of active learning concentrates on identifying the best method $P(C, i)$ to choose (C) the data points.

11.3.1 Heuristics for Active Learning

1. Start with a pool of unlabelled data $P(U, i)$.
2. Pick a few points at random and get their labels $P(C, i)$.
3. Repeat.

Some active learning algorithms are built upon Support Vector Machines (SVMs) to determine the data points to label. Such methods usually calculate the margin (W) of each unlabelled datum in $P(U, i)$ and assume W as an n-dimensional space distance commencing covering datum and the splitting hyperplane.

Minimum Marginal Hyperplane (MMH) techniques consent that the data records with the minimum margin (W) are those that the SVM is most uncertain about and therefore should be retained in $P(C, i)$ to be labelled.

Other comparable approaches, such as Maximum Marginal Hyperplane, select data with the significant W. Other approaches select a combination of the smallest as well as largest Ws.

11.3.2 Active Learning Query Strategies

Few logics for determining which data points should be labelled are:

1. Uncertainty sampling
2. Query by committee
3. Expected model change
4. Expected error reduction
5. Variance reduction

Uncertainty sampling: In this method, the active learning algorithm first tries to label the points for which the current model is least specific on the correct output.

Query by committee: A range of models are trained on the current labelled data, and vote on the output for unlabelled data; label those points for which the committee disagrees the most.

Expected model change: The algorithm first tries to label the points that would most change the existing model itself.

Expected error reduction: The algorithm first tries to label the points that would mostly reduce the model's generalization error.

Variance reduction: The algorithm first tries to label the points that would reduce output variance.

11.4 INSTANCE-BASED LEARNING (MEMORY-BASED LEARNING)

In instance-based learning (memory-based learning), the algorithm compares new problem instances with instances already seen in training, which have been stored in memory for reference rather than performing explicit generalization. It constructs hypotheses from the training instances themselves.

Computational complexity of the hypothesis can grow as $O(n)$ when the number of stored instances is n.

Examples of instance-based learning include:

- K-Nearest Neighbour (K-NN)
- Support Vector Machine (SVM)
- PCA
- Radial Basis Function (RBF) networks

11.4.1 Radial Basis Function

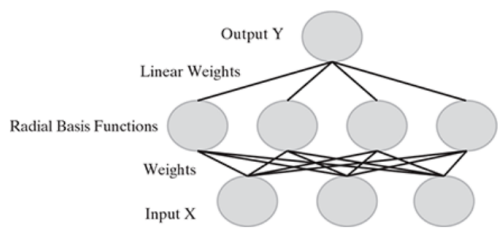


FIG. 11.4 Radial Basis Function

Figure 4: RBF

Radial Basis Function (RBF) networks typically have three layers:

- Input layer
- Hidden layer with a non-linear RBF activation function
- Linear output layer

Distance from the cluster centre is usually computed using Euclidean distance:

$$r = \|x - c_i\|$$

The most commonly used radial basis function is the Gaussian function:

$$\phi(r) = e^{-\frac{r^2}{2\sigma^2}}$$

The output is a weighted sum of hidden layer outputs:

$$y(t) = \sum w_i \phi(\|u(t) - c_i\|)$$

Advantage: Suitable for problems where the target function is highly complex.

Disadvantages:

- High cost of classification of new instances.
- Dimensionality-related problems.

11.5 ASSOCIATION RULE LEARNING ALGORITHM

Association rule learning is a method that extracts rules that best explain observed relationships between variables in data. These rules can discover important and commercially useful associations in large multidimensional data sets.

The most popular association rule learning algorithms are Apriori algorithm and Eclat algorithm.

11.5.1 Apriori Algorithm

Apriori is designed to function on a large database containing various types of transactions. It uses a bottom-up approach, where frequent subsets are extended one item at a time (candidate generation), and groups of candidates are tested against the data.

Apriori is a powerful algorithm for frequent mining of itemsets for boolean association rules.

11.5.2 Eclat Algorithm

ECLAT stands for "Equivalence Class Clustering and bottom-up Lattice Traversal." It is another frequent itemset generation method similar to Apriori.

It represents the data in vertical format using Transaction IDs (TID list).

Steps:

1. Get Transaction IDs (TID list) for each item.
2. Intersect TID lists to generate frequent itemsets.
3. Repeat the process on the conditional database.

11.6 ENSEMBLE LEARNING ALGORITHM

Ensemble means collaboration or group. Ensemble methods are models that contain many weaker models that are autonomously trained and whose forecasts are combined to create the overall prediction model.

The most popular ensemble learning algorithms are:

- Bootstrap Aggregation (Bagging)
- Boosting
- AdaBoost
- Stacked Generalization (Blending)
- Gradient Boosting Machines (GBM)
- Random Forest

11.6.1 Bootstrap Aggregation (Bagging)

Bootstrap Aggregation (Bagging) works using the principle of the bootstrap sampling method. Given a training dataset D containing m examples, bootstrap sampling draws a sample by selecting m examples uniformly at random with replacement.

Training Phase:

1. Draw a bootstrap sample from the training set.
2. Build the classifier using the bootstrap sample.
3. Repeat the process.

Classification Phase: Run all classifiers on the input and choose the class with the maximum number of votes (majority voting).

11.6.2 Boosting

Boosting is an iterative ensemble technique that decreases biasing error. If an observation is classified incorrectly, its weight is increased. Weaker learning models are trained on resampled data and the outcomes are combined using a weighted voting approach.

Adaptive Boosting (AdaBoost) is a special variant of boosting.

11.6.3 Gradient Boosting Machines (GBM)

Gradient boosting is boosting with gradient.

Gradient: Consider L as a function of an N-variable. By calculating the derivative of L, if the derivative is positive/negative, decrease/increase the variable to reduce the loss.

The gradient is the vector of derivatives of an N-variable function at the current point.

11.7 REGULARIZATION ALGORITHM

Regularization is an extension made to another method (typically regression) that penalizes models based on their complexity, favouring simpler models that generalize better.

The most popular regularization algorithms are:

- Ridge Regression
- Least Absolute Shrinkage and Selection Operator (LASSO)
- Elastic Net
- Least-Angle Regression (LARS)

Elastic Net is useful for high-dimensional datasets where correlations among independent variables result in multicollinearity. It helps in selecting correlated variable groups.

Chapter 12

Reinforcement Learning

12.2 What is Reinforcement Learning?

Reinforcement Learning (RL) is a type of machine learning in which an agent learns how to behave in an environment by performing actions and receiving feedback in the form of rewards or penalties. Unlike supervised learning, there is no labeled dataset. Instead, the system learns through trial-and-error interactions.

Trial-and-error learning involves two important strategies: exploration and exploitation. When the agent takes the known steps that have proven to be successful in yielding high reward value, then it is told to be exploiting the knowledge. On the other hand, when the agent tries out unknown path and by doing that identifies the new ways to achieve higher rewards, is termed as exploration.

The goal of reinforcement learning is to maximize cumulative reward over time.

Example:

- A robot learning to walk
- A computer learning to play chess or Go
- Self-driving cars learning navigation

A reinforcement learning system consists of several key elements that define how the learning process works.

12.3 Elements of Reinforcement Learning

12.3.1 Agent

The agent is the learner or decision-making entity that interacts with the environment. It observes the state of the environment and selects actions in order to achieve a goal.

Examples:

- Robot performing tasks
- Computer program playing a game
- Self-driving car

The agent continuously learns from experience and updates its strategy to improve performance.

12.3.2 Environment

The environment represents everything external to the agent with which it interacts. It receives the agent's actions and responds by changing its state and providing feedback in the form of rewards.

Examples:

- A game board in chess
- Road conditions for autonomous vehicles
- Physical surroundings for a robot

Based on the properties of the environment, it can be classified as

Deterministic vs. Stochastic environments: A environment is called deterministic if the outcome is known based on current state and action. For example, in a chess game, it is clear what will be the new state if a piece is moved. But if the outcome is not certain and only a probability of the possible outcomes can be estimated, then the environment is called stochastic. For example, it is not certain what will be the outcome if the dice is rolled.

Fully observable vs. Partially observable environment: When the state of the system is visible to the agent at all times, then it is called a fully observable environment. For example, in chess, the players can see the position of all the pieces on the board and thus it is a fully observable environment. But if the agent can't determine the state of the whole system all the time, then it is a partially observable environment. For example, in a game of poker, the players can’t see the cards in other players hand.

Discrete vs. Continuous environments: Within an environment, if only a finite set of actions is possible for moving from one state to another, then it is called a discrete environment. For example, the number of possible actions is limited in chess. On the other hand, if the set of actions available is infinite, then it is called a continuous environment. For example, there can be infinite possible routes for traveling from one place to another.

12.3.3 State

A state describes the current situation of the agent within the environment. It is defined as a set of information that the agent has at a given point in time. The

state is very important in a reinforcement learning context as the agent needs to plan its next set of actions based on the current situation, the reward it can get from the environment, and the goal.

Examples:

- Current position of a robot
- Board configuration in a game
- Current traffic situation for a vehicle

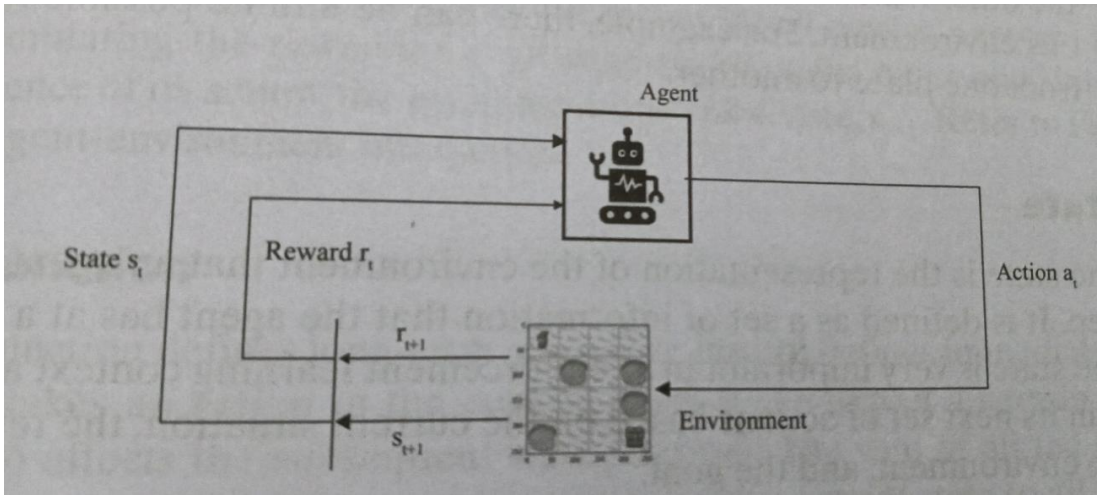


Fig. 12.1 Agent-Environment Interaction

12.3.4 Action

An action is a decision taken by the agent that influences the environment. The agent chooses actions according to its strategy or policy.

Examples:

- Move left or right
- Accelerate or stop
- Pick up an object

The collection of all possible actions is called the **action space**.

12.3.5 Policy

Policy is a strategy that the agent uses to achieve the goal. So, a policy defines the mapping from the perceived states of the environment to actions to be taken when in those states. Policy is a core component of reinforcement learning as it alone can drive the action of the agent. It can be deterministic, meaning that the environment state combination can provide the action information or it can be

stochastic, meaning that the combination will return only the probabilities of different actions. In its simplest form, a policy is a function that takes a state s as input and returns an action a. That is:

$$\pi(s) \rightarrow a$$

In this way, a policy is typically used by the agent to decide what action 'a' should be executed when it is in a given state 's? If the policy is stochastic instead of deterministic, then instead of returning a unique action 'a', the policy returns a probability distribution over a set of actions.

$$\pi(s) \rightarrow p(a)$$

For a policy-based reinforcement learning algorithm, the aim is to learn an optimal policy that achieves a specific goal.

12.3.6 Reward

A reward is a numerical value given to the agent after it performs an action. It indicates whether the action was good or bad. The effect of an action can be positive or negative, which is equivalent to reward and punishment in the real world. Through multiple trial-and-error the agent must learn to accumulate as much reward as possible and to do so it may have to alter the policy. Thus, reward becomes the key factor while creating or fine-tuning a policy.

12.3.7 Value Function

A value function defines long-term gain while reward defines immediate gain. When an agent takes an action in the current state, it can expect a certain reward. The action also affects the subsequent state's actions. The sum of all the rewards that can be accumulated by the agent starting from the current state to all the probable future states is termed as the value of the current state. In the real world, we can bring an analogy of pleasure or pain to reward, which is immediate effect, and long term happiness or sadness to value function which is the accumulation of continuous pleasure or pain over a timeline.

12.3.8 Model

A model is a replica of the environment that imitates the behaviour of the environment. This helps in deriving inferences about future courses of action based on prior knowledge about how the environment will behave in such a situation. So, an agent can identify the reward of a state and that of all probable future states without actually experiencing those situations.

The technique of solving reinforcement learning problems using models is called the *model-based approach* and the main aim of the agent is to explore the

environment to learn the model. On the other hand, the techniques that depend exploration and exploitation of the environment to gather knowledge about it and identify the maximum value or optimal policy is termed a *model-free approach.*

12.4 Reinforcement Learning Principles

12.4.1 Markov Property

The **Markov Property** is a fundamental concept in reinforcement learning and stochastic processes. It states a system is said to satisfy the **Markov Property** if the probability of transitioning to the next state depends only on the **current state and the action taken**, and not on the history of previous states. Mathematically this can be represented as:

$$P\left[S_{t+1} \middle| S_t\right] = P\left[S_{t+1} \middle| S_1, S_2, \ldots, S_t\right]$$

where S_t denotes the current state of the agent and S_{t+1} denotes the next state.

As we can see in the LHS of the equation, the transition from state S_t to S_{t+1} entirely independent of the past and thus the RHS of the equation which has all the history from S_1 to S_t, gets captured within the current state S_t, if the process has Markov Property

Transition probability: It is the probability that the agent will move from one state to another state. Drawing the similarity to moving from Markov State to S_t to S_{t+1} i.e. from the current stale to any other successor state, the state transition probability and the transition matrix are given by

$$P_{ss'} = P\left[S_{t+1} = s' \middle| S_t = s\right] = \begin{array}{c} \begin{bmatrix} P_{11} & P_{12} & \cdots & P_{1n} \\ P_{21} & P_{22} & \cdots & P_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ P_{n1} & P_{n2} & \cdots & P_{nn} \end{bmatrix} \\ \text{"}From\,State\text{"} \end{array} \quad \text{"To State"}$$

Here P_{12} represents the state transition probability of moving from state S_1 to S_2 state and so on. It should be noted that the sum of probabilities along the rows is always 1.

12.4.2 Markov Process or Markov Chain: A Markov Process, also known as a Markov Chain, is a stochastic process that satisfies the Markov Property, i.e. a

Markov Process or Markov Chain is a sequence of random states $(S_1, S_2 \ldots, S_n)$ that satisfies the Markov property. Thus, the environment can be represented using the states (S) and state transition probability matrix $(P_{ss'})$. In a discrete environment, it is called a Markov chain and in a continuous environment, it is termed a Markov process.

Example of Markov Chain

Weather Prediction Example

Suppose the weather can be either:

- Sunny (S)
- Rainy (R)

The probabilities are:

Current Weather	**Next Sunny**	**Next Rainy**
Sunny	0.7	0.3
Rainy	0.4	0.6

This system forms a Markov chain because the next day's weather depends only on today's weather.

12.4.3 Markov Reward Process: A Markov Reward Process (MRP) is an extension of a Markov Process (Markov Chain) that includes rewards associated with states. It is an important concept in reinforcement learning because it allows us to evaluate how good a state is based on the expected rewards obtained from it. It is defined by a tuple (S, P, R, γ)

where

S: set of states

P : state transition probability matrix

R: reward function

γ:discount factor

Thus, an MRP describes how rewards accumulate as the system transitions between states.

Reward: The rewards are the numerical values an agent receives based on the action taken while being in a state in the environment. The reward can be positive or negative based on the action taken by the agent.

Return: Return is the cumulative reward that the agent receives from the environment while completing a sequence of state transitions, called episodes. In RL the goal is to maximize the cumulative reward or return, rather than the reward an agent receives from the current state.

Discount Factor (γ)

The discount factor determines the importance of future rewards.

$0 \leq \gamma \leq 1$

- $\gamma=0 \rightarrow$ only immediate rewards matter
- $\gamma \approx 1 \rightarrow$ future rewards are important

12.4.4 Bellman Equation for Value Function

In reinforcement learning, the value function measures how good a state is in terms of the expected cumulative reward that an agent can obtain starting from that state.

The Bellman Equation provides a recursive relationship for the value function. It expresses the value of a state as the immediate reward plus the discounted value of the next state. This equation is fundamental in reinforcement learning and dynamic programming because it allows the value function to be computed.

The **state value function** for a policy π can be defined as:

$$v_\pi(s) = E_\pi\left[G_t \middle| S_t = s\right]$$

where

$v_\pi(s)$: value of state s under policy π

G_t: return

S_t: state at time t

The return is defined as

$$G_t = \sum_{n=0}^{\infty} \gamma^n R_{t+n+1}$$

The Bellman equation helps in finding out the optimal policies and value functions to maximize the return. For that purpose it aims to identify the optimal value function that can maximize the value compared to all other value functions.

Bellman Equation for Value Function is given by

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} P(s',r|s,a)\left[r + \gamma v_\pi(s')\right]$$

- $v_\pi(s)$ = value of state s under policy π
- $\pi(a|s)$ = probability of taking action a in state s
- $P(s',r|s,a)$ = probability of moving to state s′ with reward r
- γ = discount factor

It is important to note that the value function of the current state is now related to the value function of the successor state, which enables us to iteratively compute the value function.

Finite Markov Decision Process:

A finite Markov Decision Process is a mathematical model where the agent makes some decision or takes action based on its current state following a policy. It is a tuple of $\left(S, A, P_{SS}', R(s|a), \gamma\right)$ where:

S is the set of states

A is the set of actions the agent can take,

P_{SS}' is the transition probability matrix

$R(s|a)$ is the reward function

γ is the discount factor.

MDPs are a classical formalization of sequential decision-making problems where actions not only influence the immediate reward, but also the successor states and through those the future rewards. The goal of RL is to find an optimal policy that will achieve a high value of cumulative reward over the long run. Keeping this goal in mind, a policy π is considered better than or equal to that of π' for all states. Mathematically

$$\pi \geq \pi' \ \ iff \ \ v_{\pi}(s) \geq v_{\pi'}(s) \ \ for\ all\ s \in S$$

So, there is always at least one policy that is better than or equal to all other policies, and this policy is called optimal policy. Optimal policy(s) are denoted by:

$$v_*(s) = \max_{\pi} v_{\pi}(s), \ \ for\ all\ s \in S$$

In view of this, the Bellman Optimality equation for v_* is given by

$$V(s) = \max_{a} \sum_{s',r} P(s',r|s,a)\left[r + \gamma v_{\pi}(s')\right]$$

12.5 Reinforcement Learning Algorithm

12.5.1 **Dynamic Programming**

Dynamic Programming (DP) refers to a class of algorithms used in Reinforcement Learning to compute optimal policies and value functions when the complete model of the environment is known. These methods are mainly used for solving Markov Decision Process (MDP) problems. The key idea is to search for the optimal policy by using the value functions where the problem is broken down into multiple sub problems and the result of a sub-problem is used in similar sub problems, instead of independently solving those. To be solved through a dynamic programming approach, a problem should have the following properties:

a) we should be able to divide the problem into similar sub problems
b) if we can find the optimal solution for a sub-problem, then that solution can be used to find the optimal solution for the overall problem.

The concept of dynamic programming was introduced by Richard Bellman, and relies on recursive relationships known as the Bellman Equation.

In Dynamic Programming, the key concept is to first identify the value function that will satisfy the Bellman equation and then using the optimal value function identify the optimal policy. The process is logically divided into three steps:

1. Policy Evaluation/Prediction
2. Policy Improvement
3. Policy Iteration/Control

12.5.1.1 Policy Evaluation/Prediction*:* Policy evaluation (also called prediction) is the process of computing the state-value function $v_{\pi}(s)$ for a given policy π. The value function represents the expected return starting from

state s and following policy π thereafter. Policy evaluation is based on the Bellman Equation, which defines a recursive relationship for the value function:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} P(s',r|s,a)\left[r + \gamma v_\pi(s')\right]$$

Since solving the Bellman equation directly is difficult, iterative policy evaluations are used and the steps for this are as follows:

1. Initialize $v_\pi(s)$ arbitrarily (often zero).
2. Update the value of each state using the Bellman equation.
3. Repeat the update for all states.
4. Continue until the value function converges.

This process gradually improves the estimate of the state values.

12.5.1.2 Policy Improvement*:* Policy Improvement is an important concept in Reinforcement Learning used to obtain a better policy from an existing policy. It is typically applied after policy evaluation in solving a Markov Decision Process (MDP). The goal of policy improvement is to modify the current policy so that it selects actions that yield higher expected returns according to the value function. After evaluating a policy π, we know the value of each state $v_\pi(s)$. Using these values, we can choose actions that maximize the expected reward. If a new policy π′ produces higher or equal value for all states compared to π, then π′ is considered an improved policy.

Policy improvement is based on the Bellman Optimality Principle, introduced by Richard Bellman.

The improved policy is obtained by selecting the action that maximizes the expected return:

$$\pi'(s) = \arg\max_a \sum_{s',r} P(s',r|s,a)\left[r + \gamma v_\pi(s')\right]$$

So, the Policy Improvement Theorem states that:

If the new policy is greedy with respect to the value function of the current policy, then the new policy will be at least as good as the previous policy.

12.5.1.3 Policy Iteration/Control: Policy Iteration is a dynamic programming algorithm that finds the optimal policy by repeatedly performing policy evaluation and policy improvement.

Steps of Policy Iteration

1. Initialize a policy π arbitrarily.
2. ***Policy Evaluation***

   Compute the value function $v_\pi(s)$ for the current policy.
3. ***Policy Improvement***

   Update the policy by choosing actions that maximize expected return.
4. Repeat the evaluation and improvement steps until the policy no longer changes.

When the policy becomes stable, the optimal policy has been found.

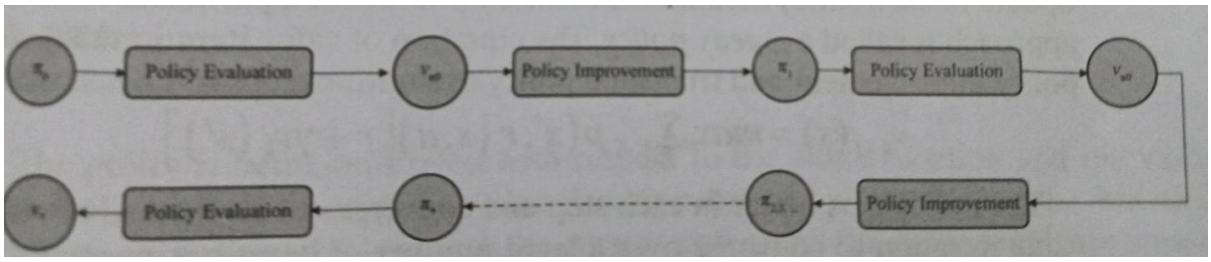


Fig. 12.2: Policy Iteration

12.5.1.4 Value Iteration

Value Iteration is a simplified version of policy iteration that combines policy evaluation and policy improvement into a single step.

Instead of evaluating a policy completely, it directly updates the value function using the Bellman optimality equation where the value iteration update rule is given by:

$$V(s) = \max_a \sum_{s',r} P(s',r|s,a)\left[r + \gamma v_\pi(s')\right]$$

Steps of Value Iteration are as follows:

1. Initialize V(s).
2. Update values using the value iteration formula.
3. Repeat updates until values converge.
4. Extract the optimal policy from the final value function.

12.5.1.5 Generalized Policy Iteration:

Generalized Policy Iteration (GPI) is the term that is used to refer to the idea of letting the policy evaluation and policy-improvement processes interact,

independent of the granularity of the two processes. Almost all reinforcement learning methods can be described as GPL.

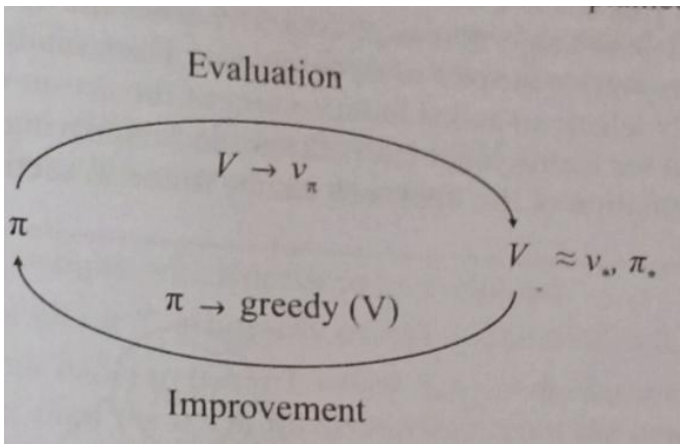


Fig. 12.3: Generalized Policy Iteration

The policy is being improved with respect to the value function and the value function is driven toward the value function for the policy. The value function stabilizes once it is consistent with the current policy and the policy stabilizes once it is greedy for the current value function. Thus, the cycle stops once both the processes stabilize, as it reaches the Bellman optimality equation and the policy and the value function are optimal. In GPI, the two processes together achieve the optimality goal though neither of them is attempting to achieve it directly.

12.5.2 Monte Carlo Methods

In the previous section, we discussed solving RL problems for MDPs, where we assumed that we have full knowledge of the environment, like the complete probability distribution of all the possible transitions, the reward function, etc. which was the basis for using dynamic programming. But in the real world we may not have the full knowledge of the environment and a new class of RL called model-free learning comes into the picture there. Monte Carlo Methods are one such class of model-free algorithms. In essence, Monte Carlo methods are learning methods that require only experience - sample sequences of states, actions, and rewards from actual or simulated interactions with the environment. The methods work by averaging sample returns. The key assumption is that the experience is divided into episodes and that all episodes terminate at the end irrespective of the actions taken. The value estimates and policies change only after the completion of an episode. The agent takes action and obtains rewards during an episode and we take an average of the rewards when the episode terminates.

$$V(S_t) = V(S_t) + \alpha\left[G_t - V(S_t)\right]$$

where G_t is actual return following time t and α is a constant step-size parameter.

Epsilon-Greedy Action Selection: In reinforcement learning, an agent must decide whether to exploit, i.e. choose the action with the highest estimated value or to explore, i.e. to try different actions to discover potentially better rewards. But the ε-greedy strategy solves this problem by selecting actions probabilistically. It is a strategy used in Reinforcement Learning to balance exploration and exploitation while learning an optimal policy. According to the ε-greedy method:

- With probability $1-\varepsilon$, the agent selects the best action (highest estimated value).
- With probability ε, the agent selects a random action.

Thus, the agent explores occasionally while mostly choosing the best-known action.

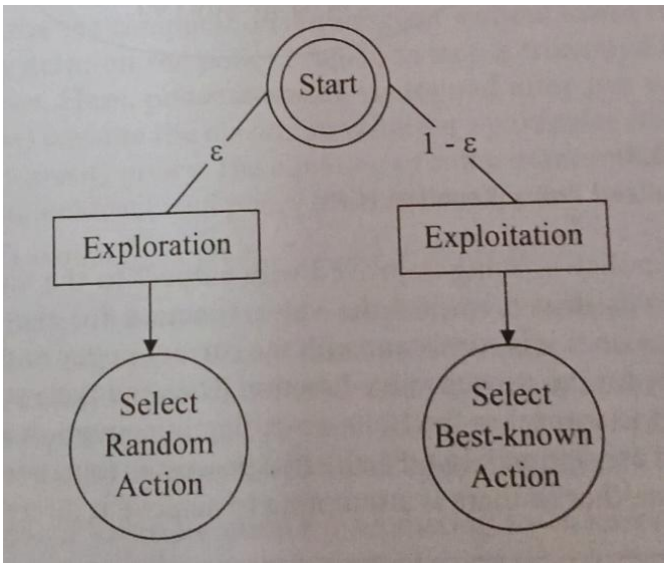


Fig. 12.4: Epsilon-greedy action selection

12.5.3 Temporal Difference Learning

One of the most effective learning processes in RL is Temporal Difference (TD) learning, which combines the ideas of Monte Carlo methods and DP methods. Similar to Monte Carlo methods, TD methods do not depend on the knowledge of the environment's dynamics or the model and thus learn directly from the experience. Also, similar to DP, TD methods update the estimates based in part on other learned estimates and do not wait for the final outcome.

12.5.3.1 TD Prediction: Similar to Monte Carlo methods, TD methods use experience to solve prediction problems. But the main difference in approach is that the Monte Carlo methods wait until an episode completes and the return is known, and then use that return to update the estimate for $V(S_t)$; whereas, TD methods need to wait only until the next time step. The update is done immediately after transitioning to S_{t+1} and receiving R_{t+1} at t+1, using the observed reward R_{t+1} and the estimate $V(S_{t+1})$:

$$V(S_t) = V(S_t) + \alpha\left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right]$$

There are two different approaches to implement TD prediction methods- on policy and off policy.

12.5.3.2 On Policy TD Control-SARSA

Here we start by learning the action-value function rather than learning the state value function. The aim is to estimate $q_\pi(s,a)$ for the current behaviour policy π for all state s and actions a. So, instead of learning just the values of the states by transitioning from state to state, we will learn the values of the state-action pairs transitioning from state-action pair to state-action pair. The TD prediction equation of will get updated to:

$Q(S_t, A_t) = Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right]$The update is made after each transition from non-terminal states. If S_{t+1} is terminal then $Q(S_{t+1}, A_{t+1})$ is defined as zero. You can see that the above equation uses every element of the group of events, $(S_t, \mathrm{A}_t, \mathrm{R}_{t+1}, S_{t+1}, \mathrm{A}_{t+1})$ [(State-Action-Reward-State-Action)] which has driven the algorithm's name - SARSA.

12.5.3.3 Off Policy TD Control-Q-learning

In 1989 Chris Watkins developed an off-policy TD control algorithm called *Q*-learning which significantly simplified the algorithm and enabled early convergence proofs. The driving equation is:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_a Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right]$$

As it is evident from the equation, the optimal action-value function q* is obtained directly from the learned action-value function Q, independent of the policy being followed. The policy only of the policy being plays a role in determining which state-action pairs are the visited algorithm and updated, but as long as it is ensured that all pairs continue to be updated. means that the

converges correctly. This is a model-free learning process, which agent explores the environment and learns directly from it, without constructing an internal model or MDP. The Agent only knows about the possible states and actions in an environment at the beginning and then discovers the state transitions and rewards by exploration.